

Visualisation & Web App

Visualisation de données avec Python

Xavier Gendre 

Avant-propos



Nous parlerons de ...

- Représentations standards
- Principes de “bonne” visualisation
- Exemples d’implémentations avec Python



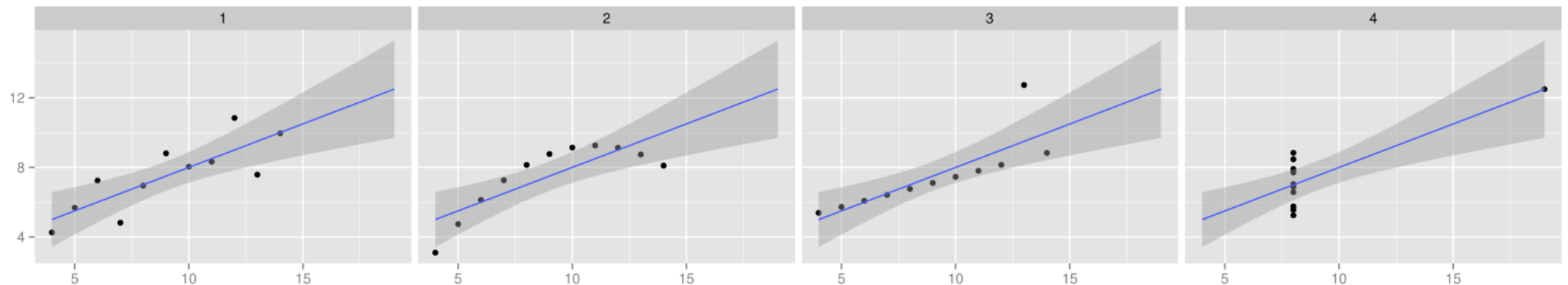
Nous ne parlerons pas de ...

- Infographie
- Aspects cognitifs de la perception

Visualisation de données

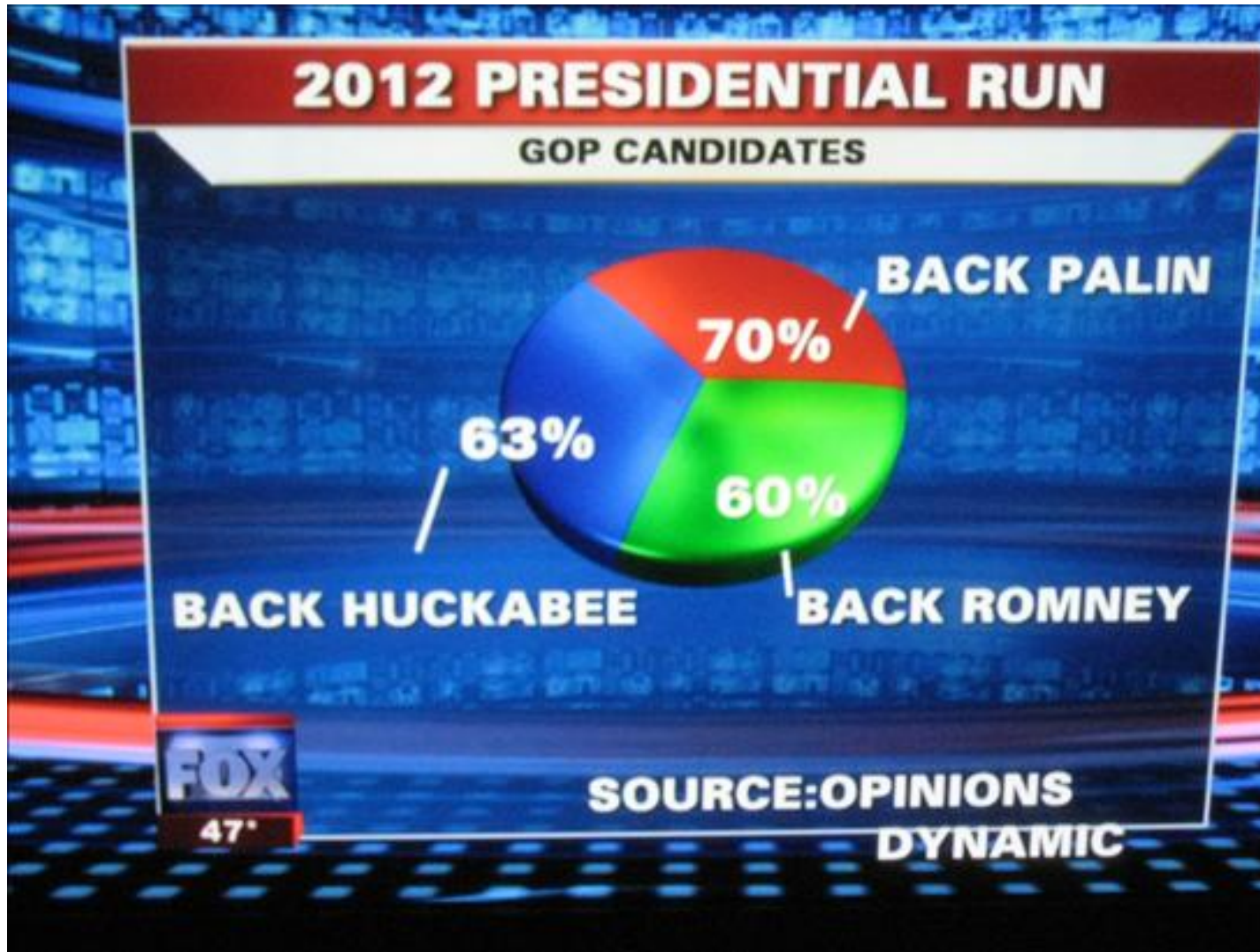
La visualisation de données (ou *data visualization*) peut :

- aider à la compréhension des “patterns” des données,
- détecter des structures cachées dans les données,
- fournir un simple résumé.



Mauvaise visualisation (1)

Sans commentaire 🙄



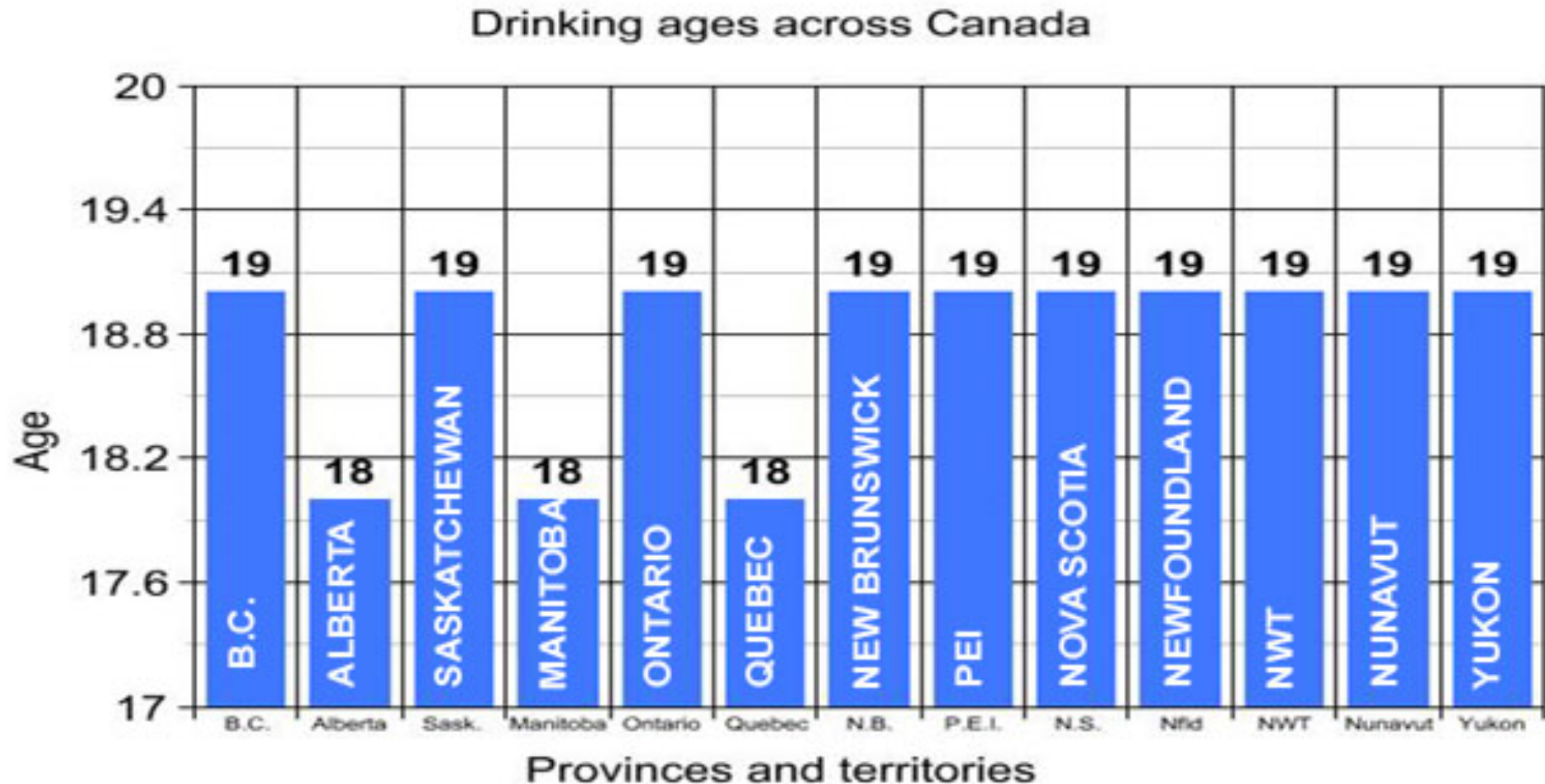
Mauvaise visualisation (2)

En France aussi 🙄



Mauvaise visualisation (3)

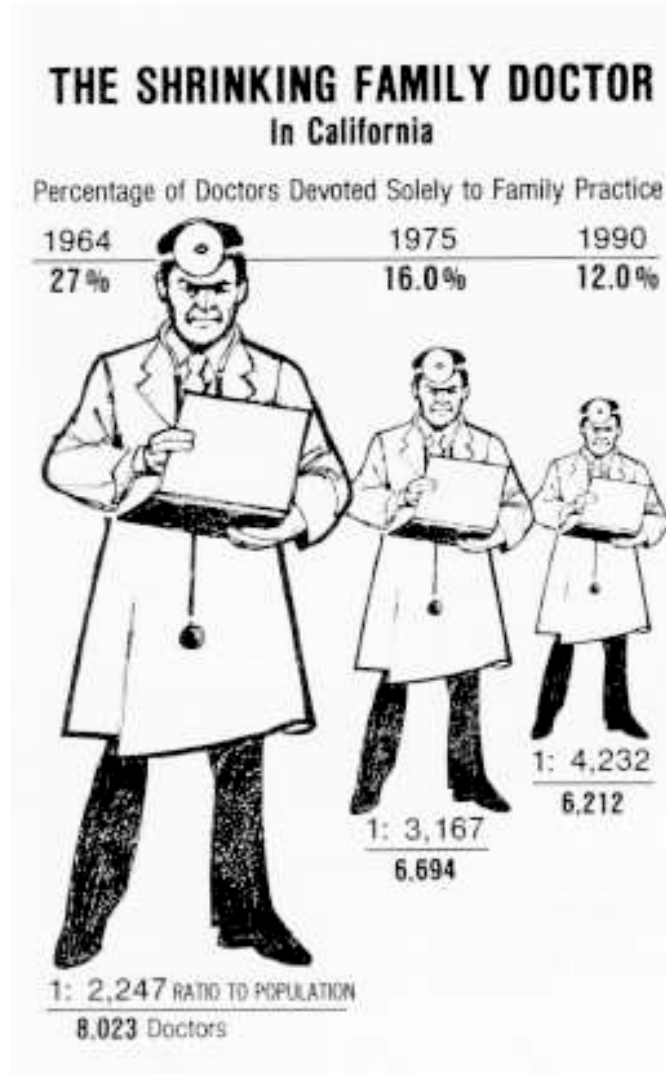
Visualisation non informative



Canadian Centre on Substance Abuse

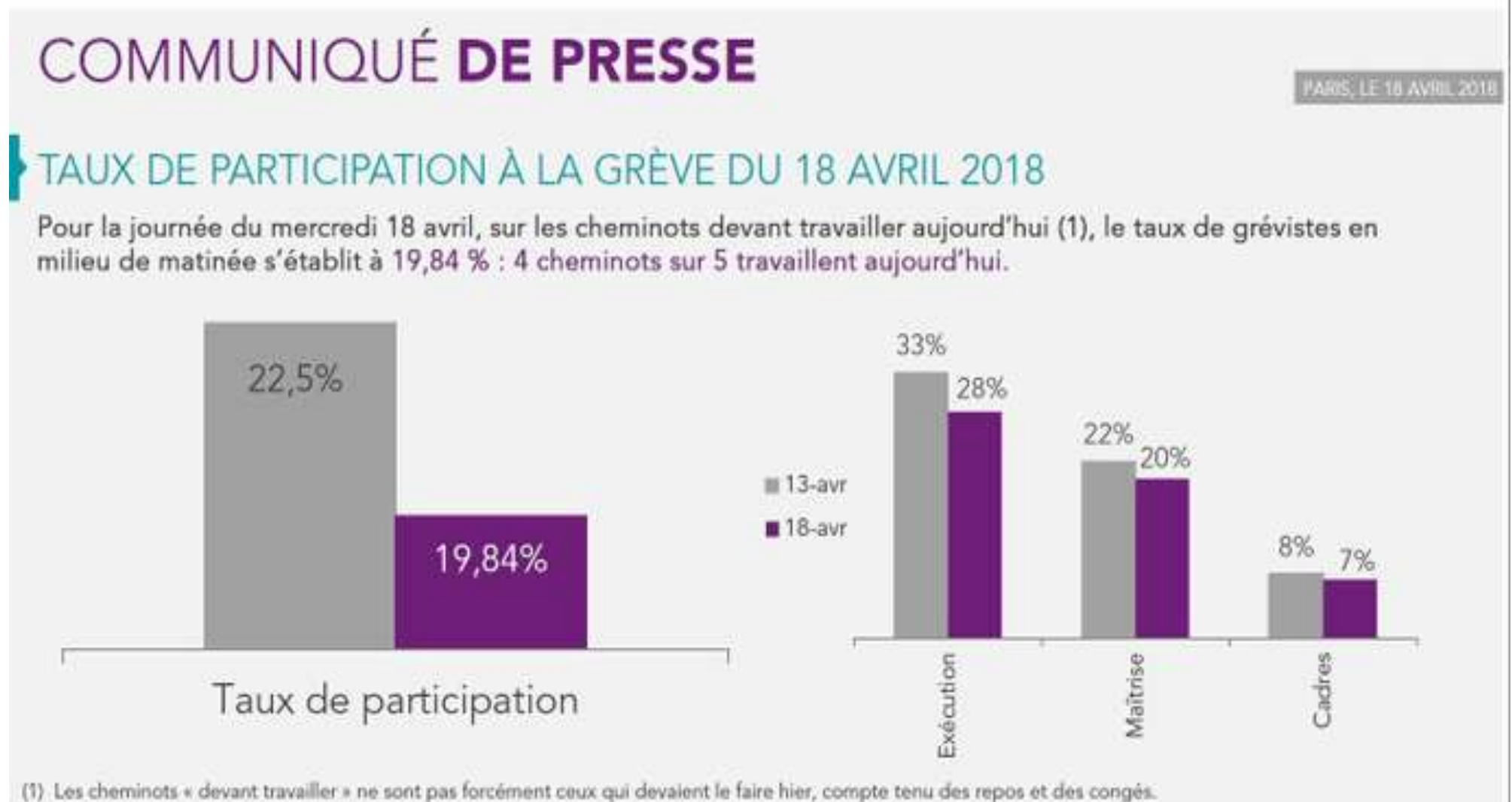
Mauvaise visualisation (4)

Facteur d'échelle faux



Mauvaise visualisation (5)

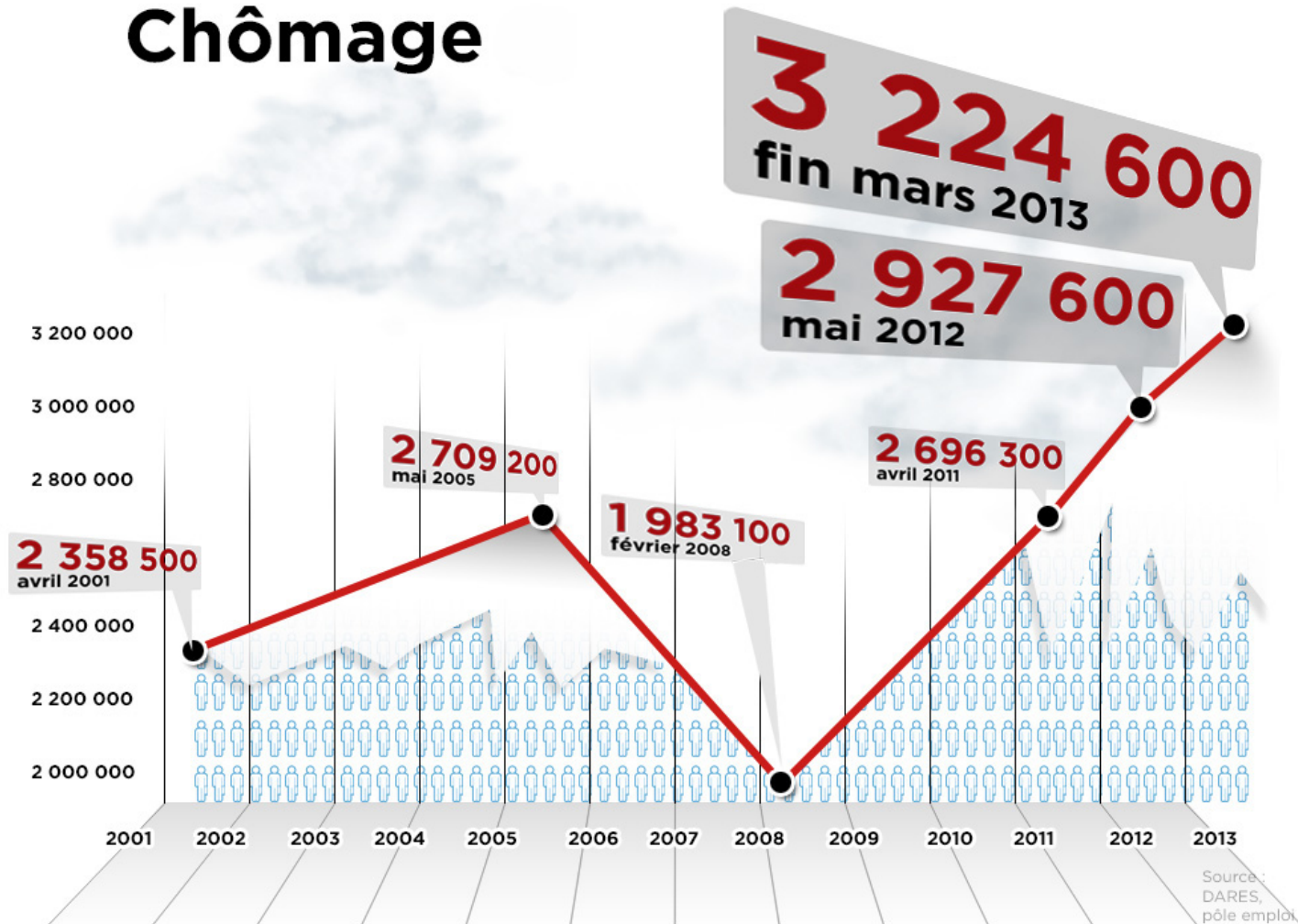
Origine des axes



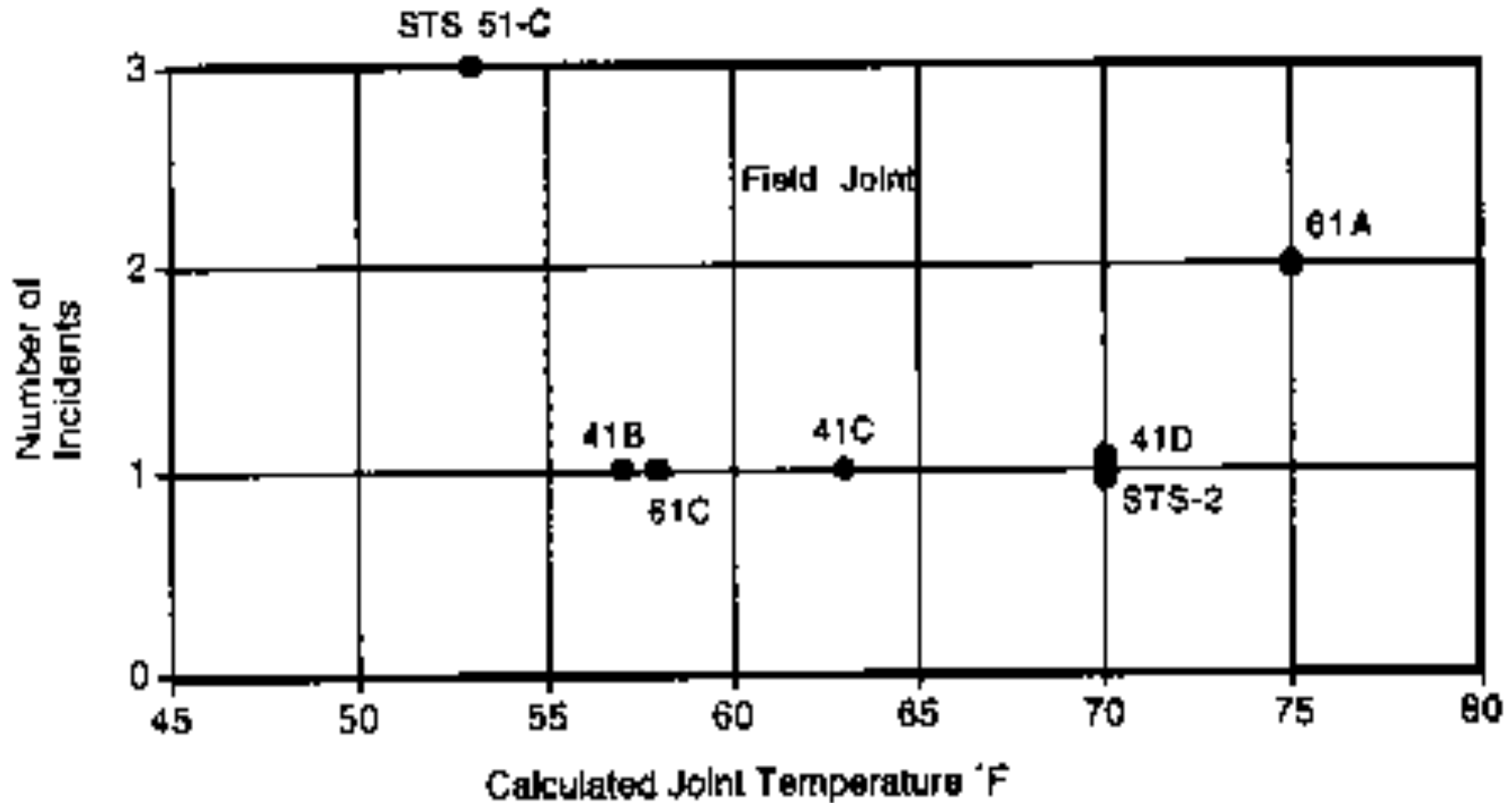
Mauvaise visualisation (6)

Que laisse-t-on penser ?

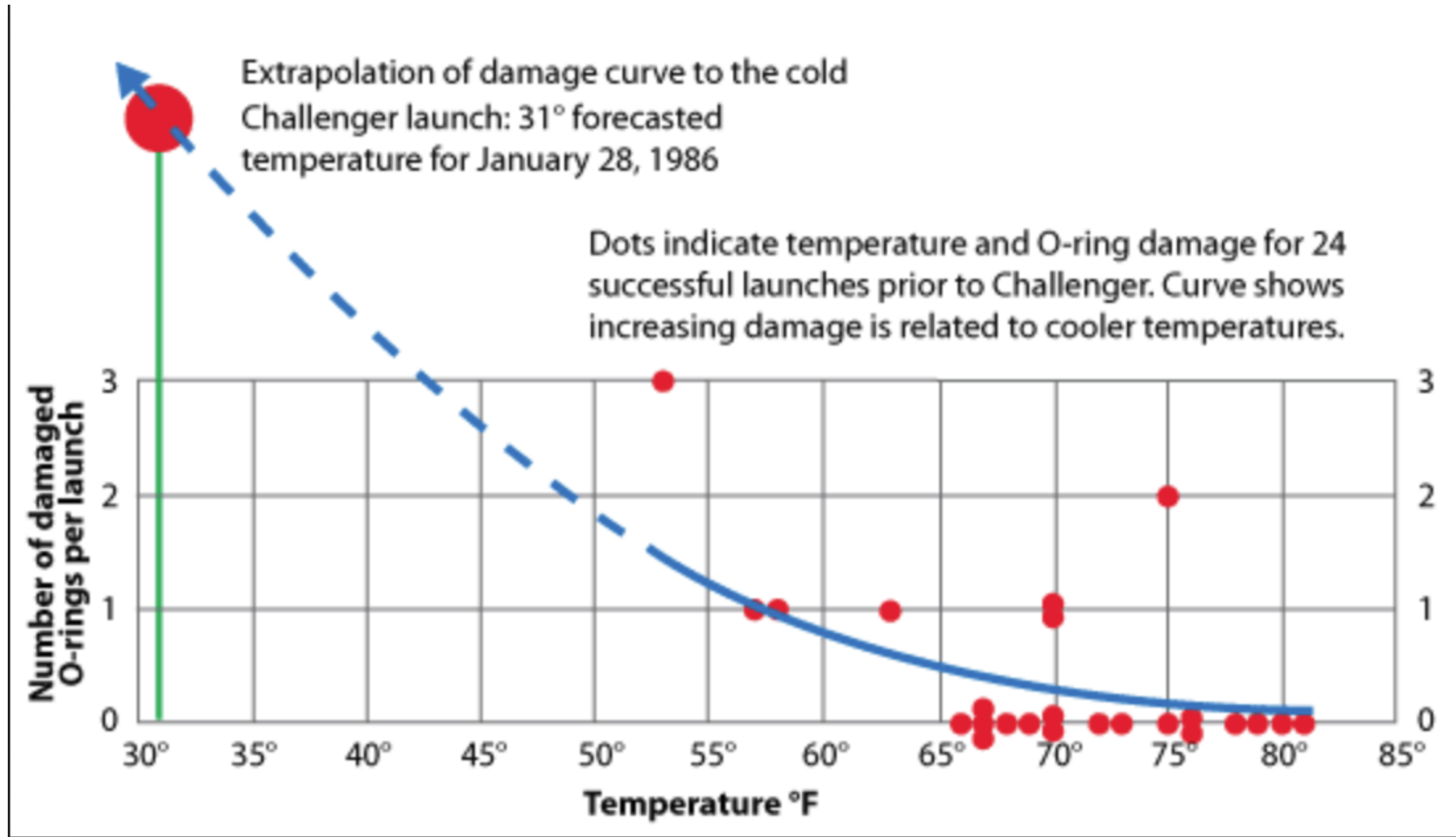
Chômage



Mauvaise visualisation (7)

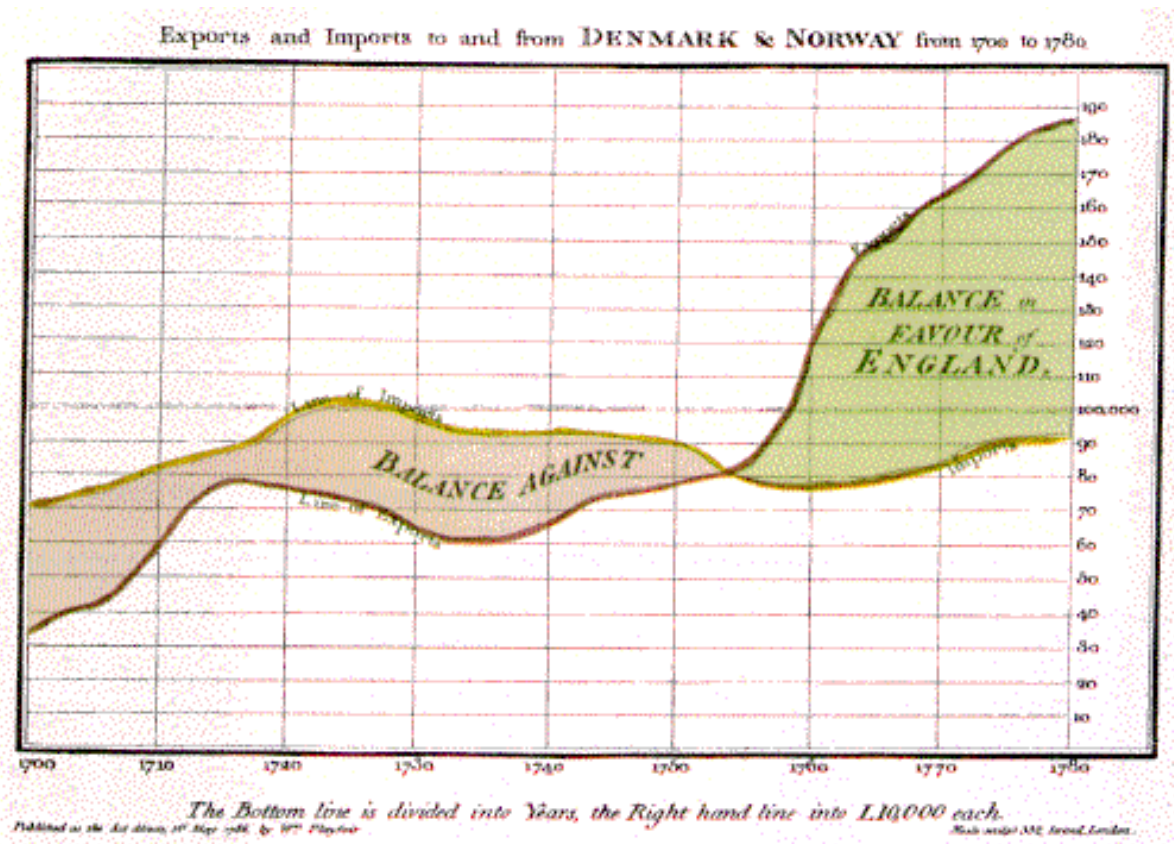


Accident de la navette spatiale Challenger, 1986



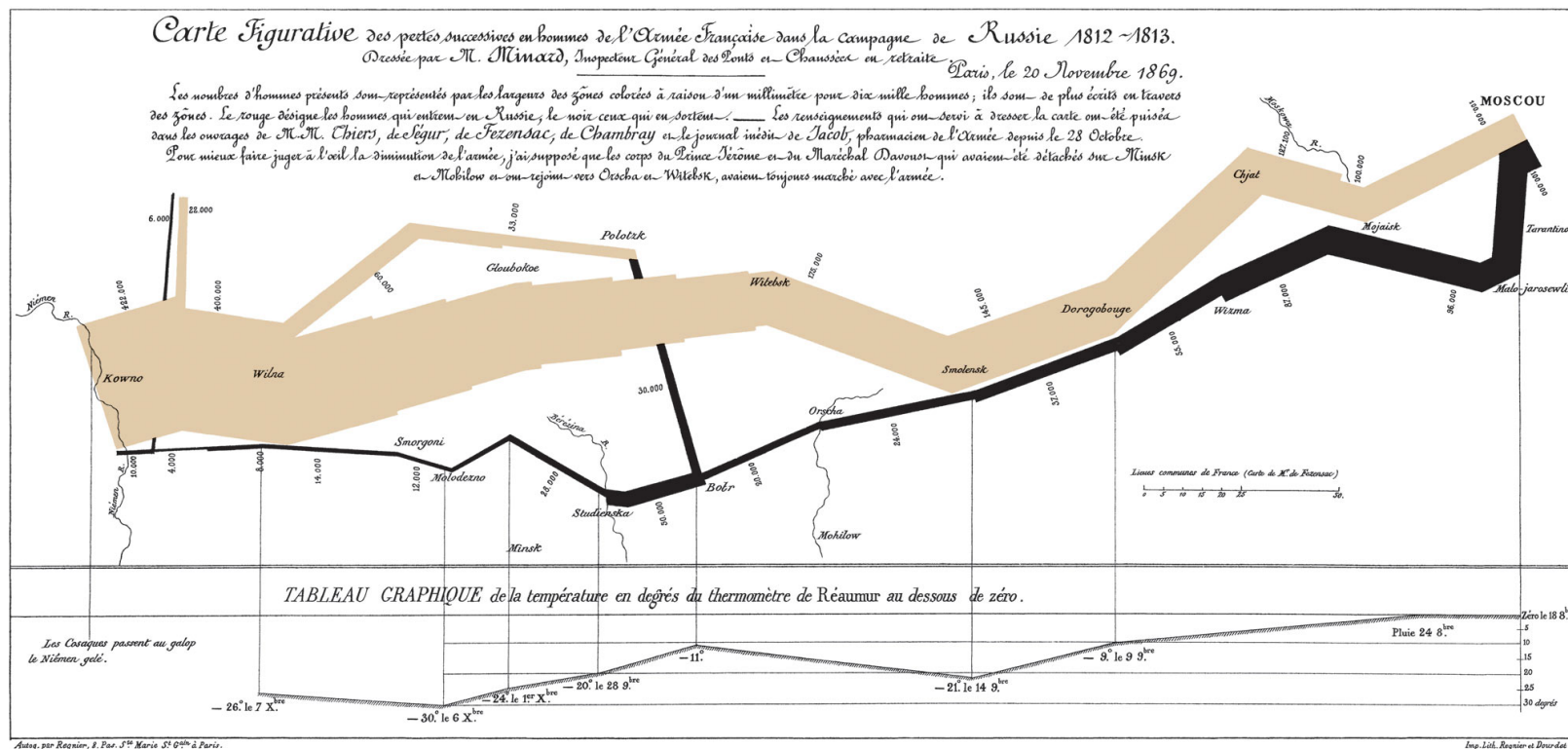
William Playfair

William Playfair (1759-1823) est considéré comme l'inventeur des formes communes de graphiques pour représenter les données (*line plots*, *bar chart* et *pie chart*).



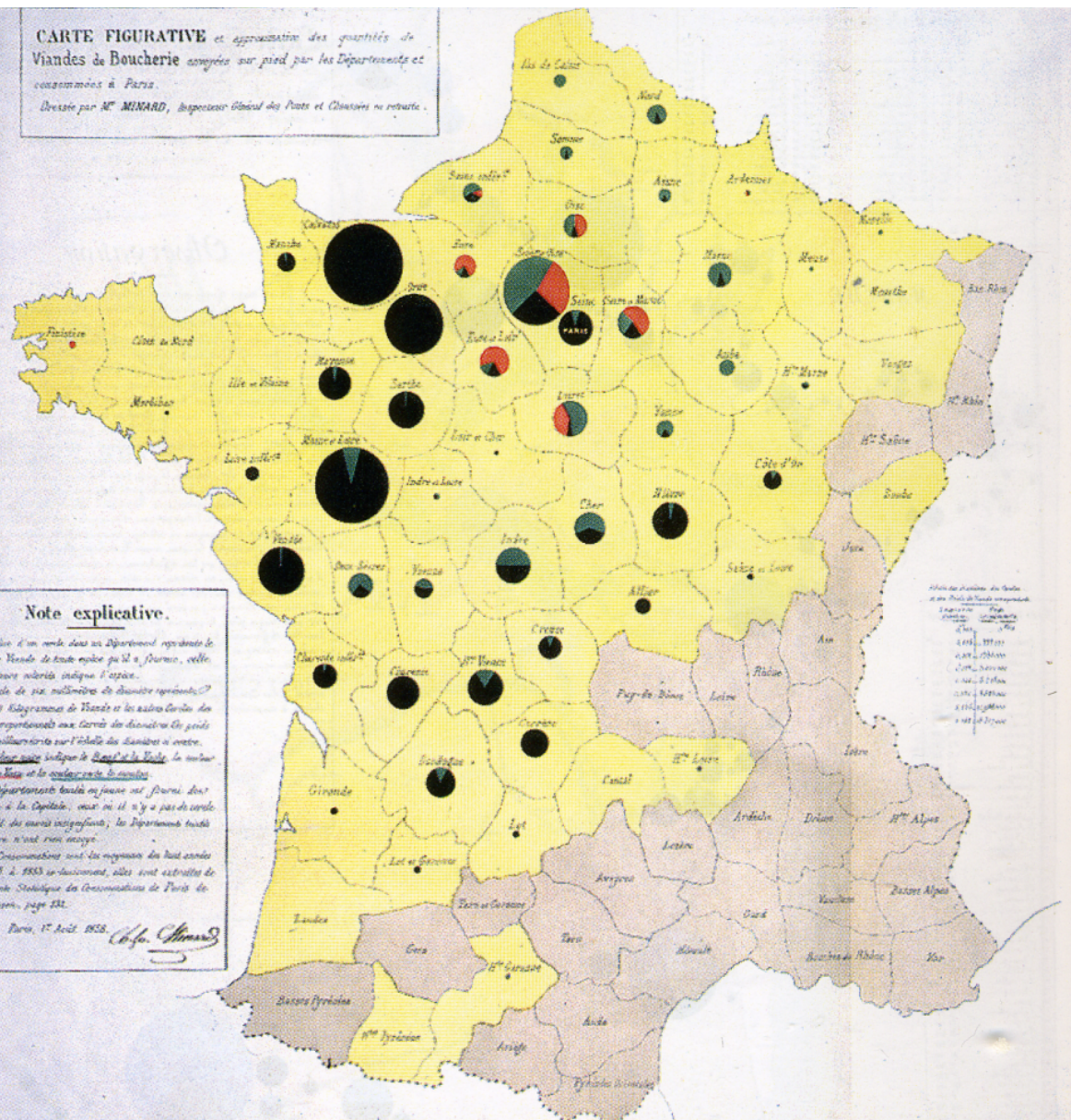
Charles Minard

Charles Minard (1781-1870) a fortement contribué au domaine de l'information graphique et statistique en particulier avec l'utilisation de cartes.



CARTE FIGURATIVE et approximation des quantités de
Viandes de Boucherie consommées sur pied par les Départements et
consommées à Paris.

Dressée par M. MINARD, Inspecteur Général des Pêches et Chaussées en retraite.



Florence Nightingale

Florence Nightingale (1820-1910) est connue comme étant la “mère” de la profession d’infirmière moderne. Elle a aussi contribué aux représentations graphiques.

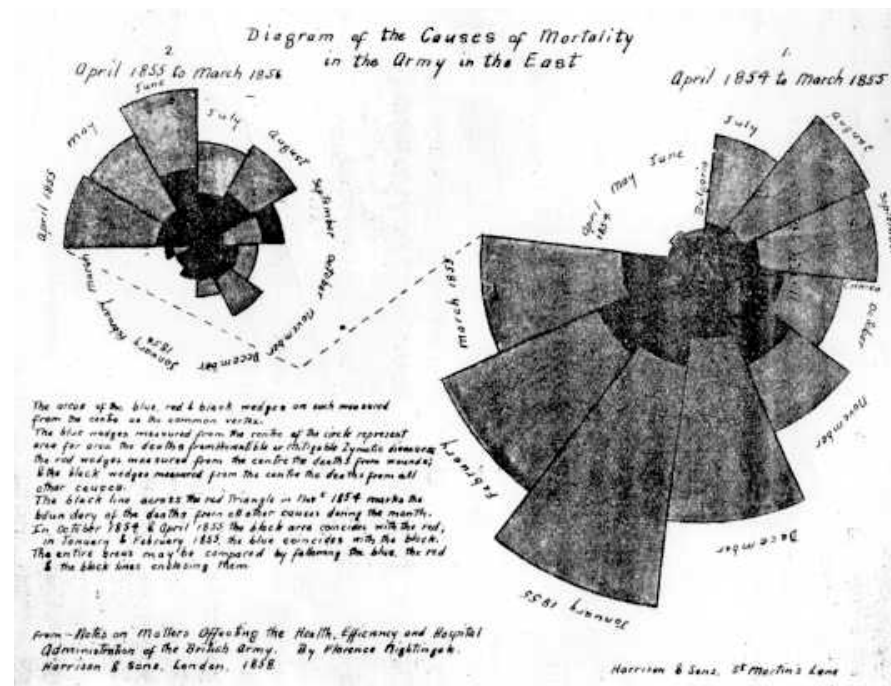


FIG. 8 — The type of graph used by FLORENCE NIGHTINGALE in 1858 to which she gave the name Coxcomb.

John Snow

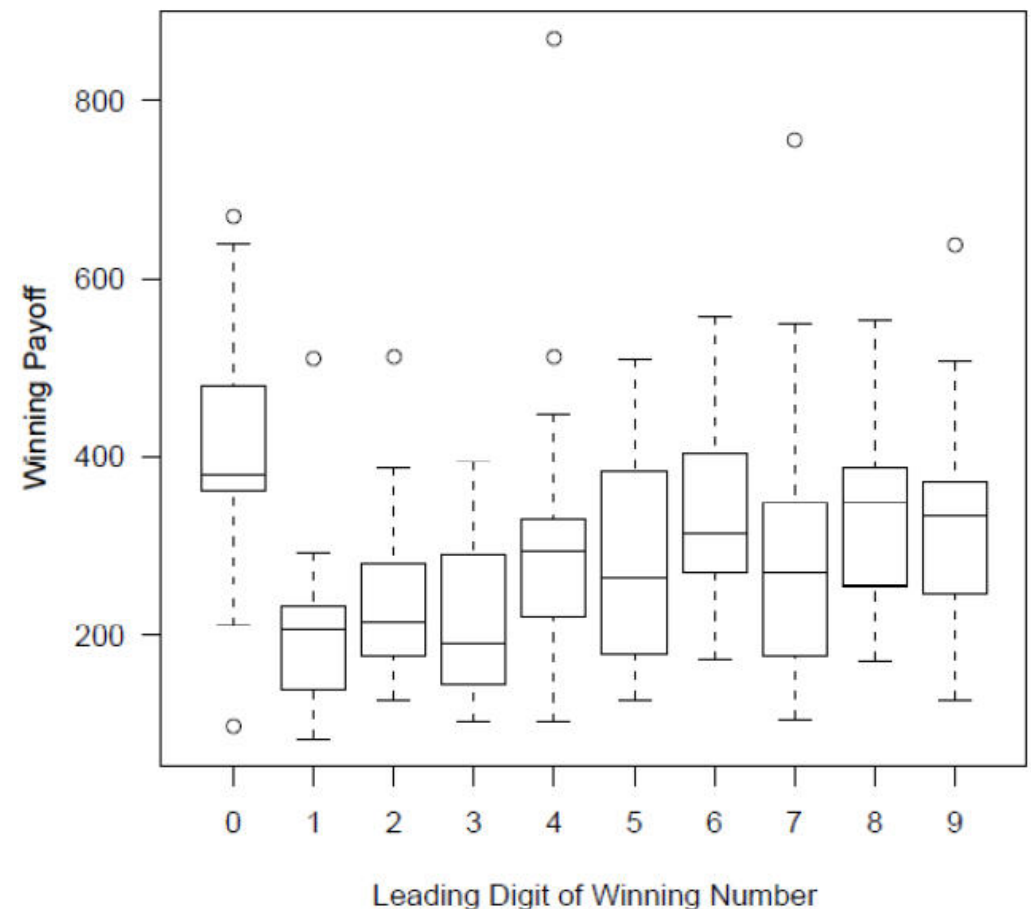
John Snow (1813-1858) est connu pour avoir tracé (et détecté) les sources de Choléra à Londres.



Ronald Fisher et John Tukey

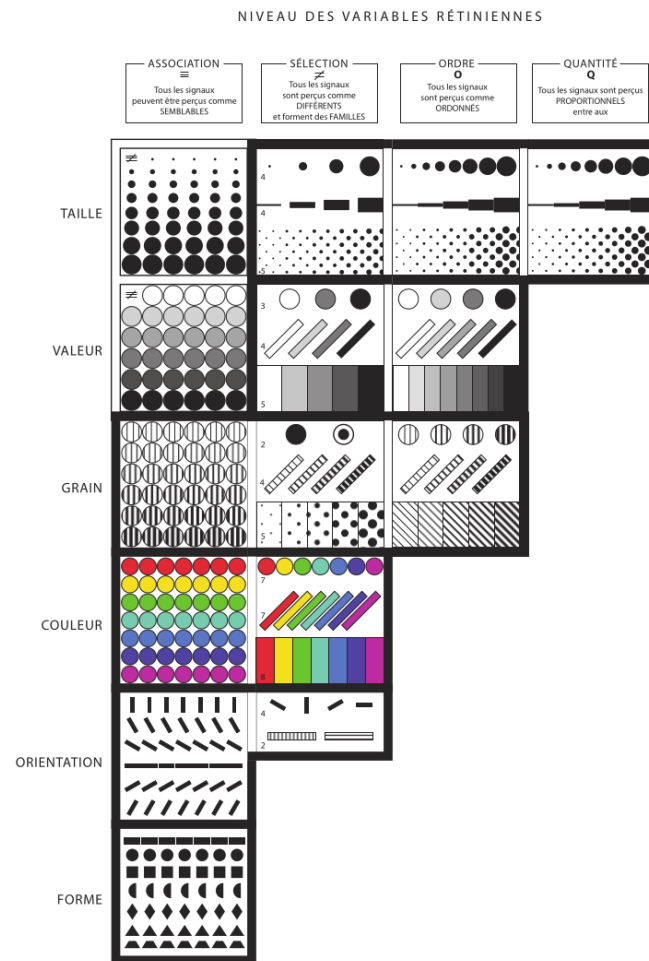
Ronald Fisher (1890-1962) et John Tukey (1915-2000) ont introduit des méthodes graphiques avancées pour l'analyse des données.

- Fisher : dessin des données pour comprendre les relations,
- Tukey : promotion de l'analyse de données exploratoires, il a créé en particulier le *box plot*, le *stem plot* et le *leaf plot*.



Jacques Bertin

Jacques Bertin (1918-2010) est l'auteur de la *sémiologie graphique*, un système de signes pour la transmission de l'information.



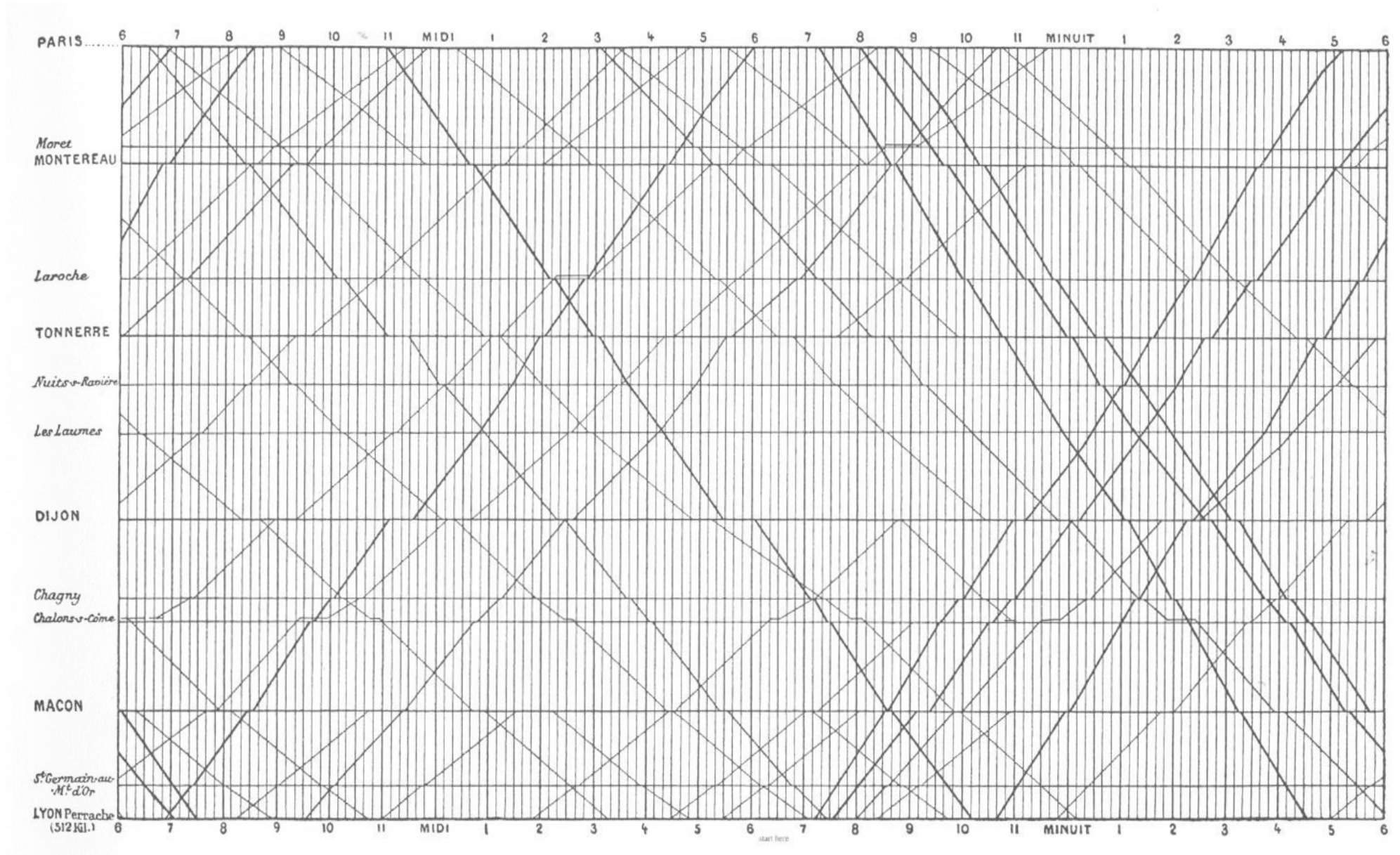
Edward Tufte

Edward Tufte (1942-) a rédigé *The Visual Display of Quantitative Information*.



Orange and green colors correspond to states where support for vouchers was greater or less than the national average. The seven ethno-religious categories are mutually exclusive. "Evangelicals" includes Mormons as well as born-again Protestants. Where a category represents less than 1% of the voters of a state, the state is left blank.

L'importance de l'aspect mais pas à n'importe quel prix !



Example sparklines in small multiple



Example sparklines of the [Dow Jones Industrial Average](#) and [S&P 500](#) on February 7, 2006

Tufte a développé et popularisé de nombreux principes :

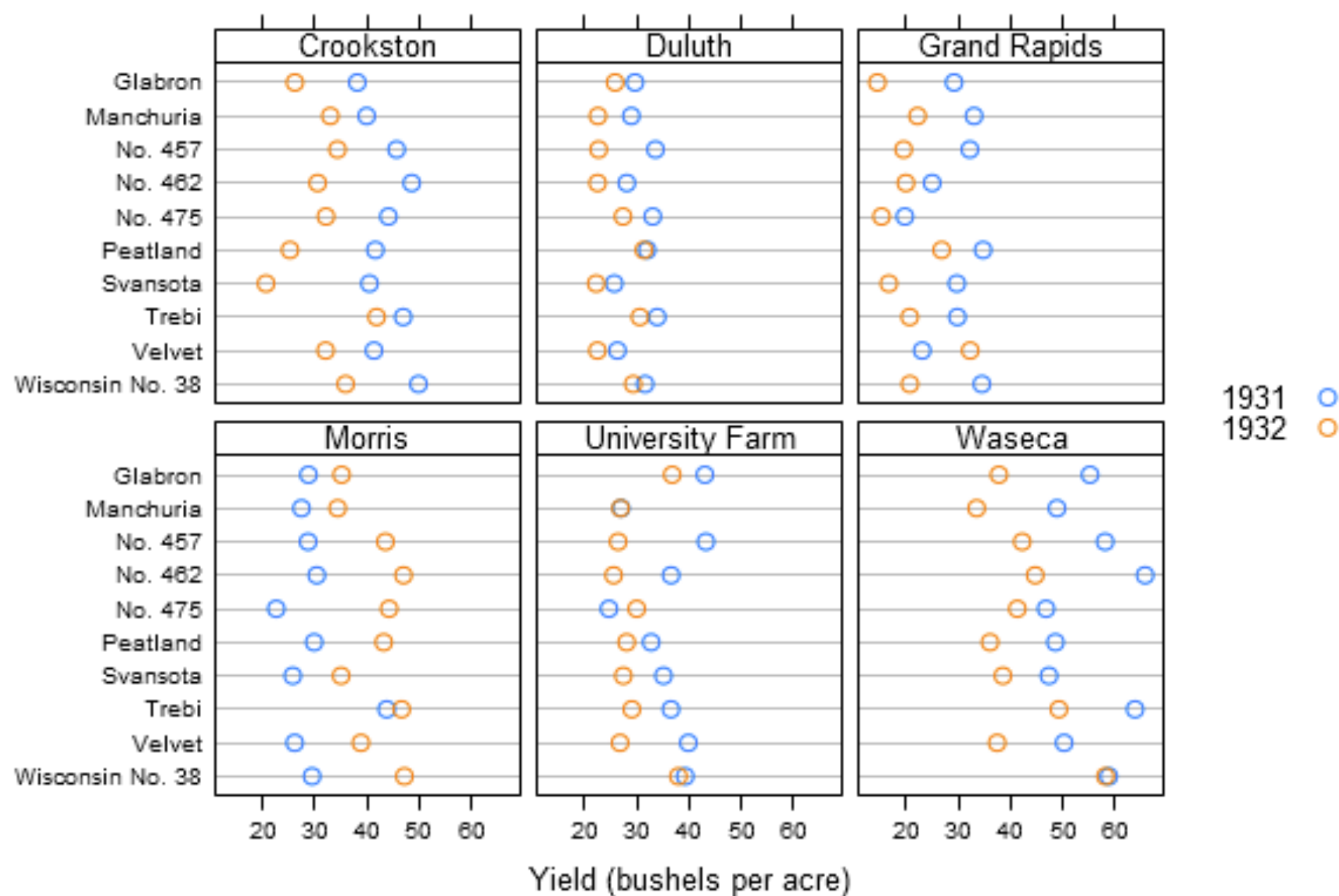
- *Graphics reveal data* : show the data without distorting it, “above all else show the data”
- *Small multiple* : understanding one slice makes understanding others easier
- *Lie factor* : effect shown/effect in reality
- *Graphical integrity* : no lies, let data vary, not design
- *Data density* : maximize data/ink ratio
- *Sparklines* : seems they haven’t caught on
- *Chartjunk* : self-explanatory

William Cleveland

William Cleveland (1943-) a écrit *Elements of Graphing Data and Visualizing Data*.

Cleveland est connu pour promouvoir le *dot plot* comme alternative aux barres, camemberts, ... car il permet une certaine clarté et une facilité de comparaison des données.

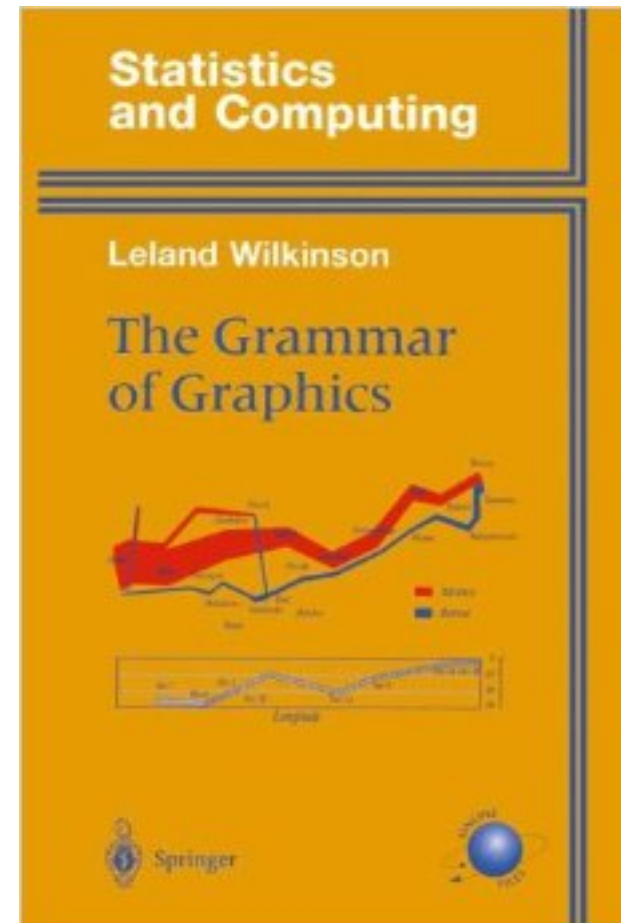
Cleveland est aussi un pionnier dans les treillis et les comparaisons de panels.



Leland Wilkinson

Leland Wilkinson (1945-2021) a écrit *The Grammar of Graphics*. Il a eu une influence importante sur la façon de penser les graphes :

- Règles mathématiques et esthétiques.
- Au-delà de l'esthétique d'un contenu statique.
- Adaptée aux graphiques dynamiques qui demandent une réflexion plus importante pour pouvoir zoomer, flouter, ...



Theme
Coordinates
Statistics
Facets
Geometries
Aesthetics
Data



- *Data* : variables à afficher,
- *Aesthetics* : dimensions de représentation des données,
- *Geometries* : formes utilisées pour représenter les données,
- *Facets* : tableau (lignes et colonnes) de graphes,
- *Statistics* : modèles ou transformations statistiques des données,
- *Coordinates* : espace de représentation (horizontal, vertical, cartésien, polaire),
- *Scales* : échelle des axes (linéaire, logarithmique, à l'envers), les couleurs de remplissage,
- *Theme* : description de l'arrière plan.

Visualisation avec Python

Les principaux modules :

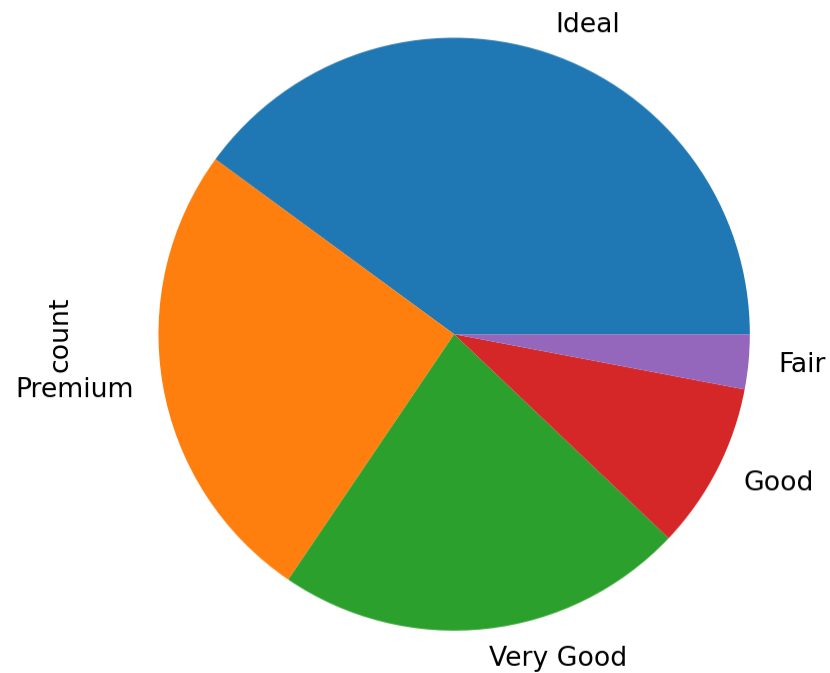
- [Matplotlib](#) : le plus connu, il est inspiré des graphiques produits par le langage *MATLAB*. Les graphiques sont simples mais le module est très polyvalent (e.g. Pandas).
- [Bokeh](#) : dédié à la visualisation interactive pour le web, il est basé sur le livre *The Grammar of Graphics* de Leland Wilkinson. Ce module puissant est plus exigeant.
- [Plotly](#) : également orienté visualisation interactive, il est plus accessible et se révèle un bon choix pour les applications web et les dashboards interactifs.

Mais aussi ...

- [Seaborn](#) : il étend Matplotlib en simplifiant la syntaxe pour produire des graphiques plus “modernes”. Moins flexible, il demande de connaître Matplotlib.
- [hvPlot](#) : ce module fait partie de l'écosystème [Holoviz](#) et joue le rôle d'intermédiaire avec les principaux modules de visualisation.
- [PyViz](#) : module riche orienté visualisation de données.
- [Vega-Altair](#): module plus simple d'utilisation pour des visualisations courantes.
- ...

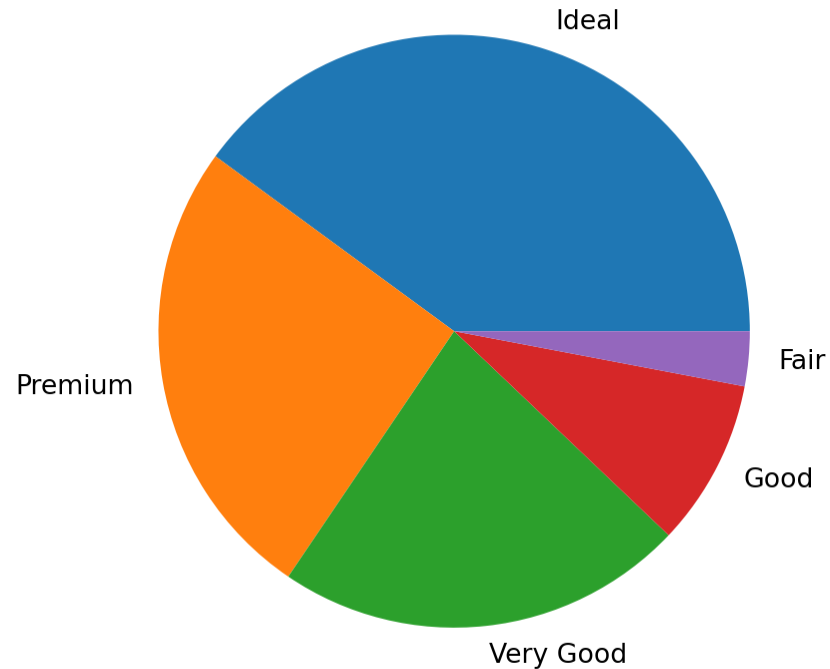
Pie chart (camembert) - Pandas

```
1 import pandas as pd
2
3 diamonds = pd.read_csv("data/diamonds.csv")
4 counts = diamonds.cut.value_counts()
5 counts.plot.pie(figsize=(6, 5));
```



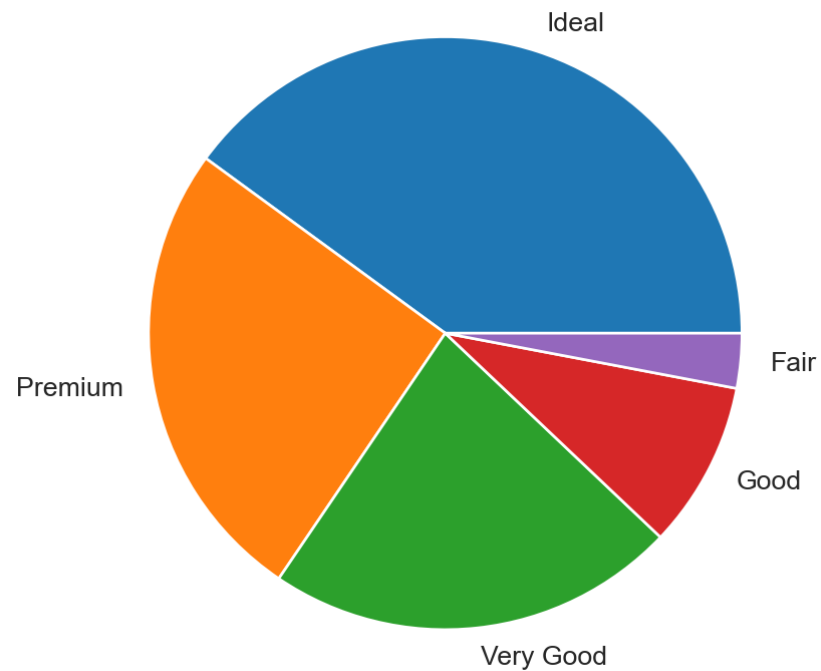
Pie chart (camembert) - Matplotlib

```
1 import matplotlib.pyplot as plt
2
3 fig, ax = plt.subplots(figsize=(6, 5))
4 ax.pie(counts, labels=counts.index);
```



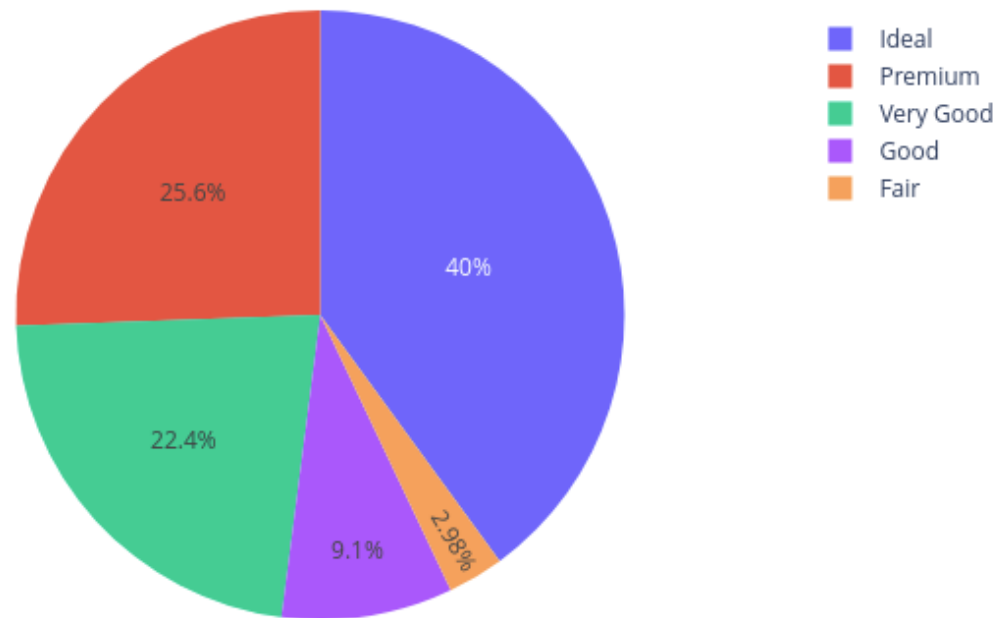
Pie chart (camembert) - Seaborn

```
1 import seaborn as sns
2
3 sns.set_style("whitegrid")
4 fig, ax = plt.subplots(figsize=(6, 5))
5 ax.pie(counts, labels=counts.index);
```



Pie chart (camembert) - Plotly

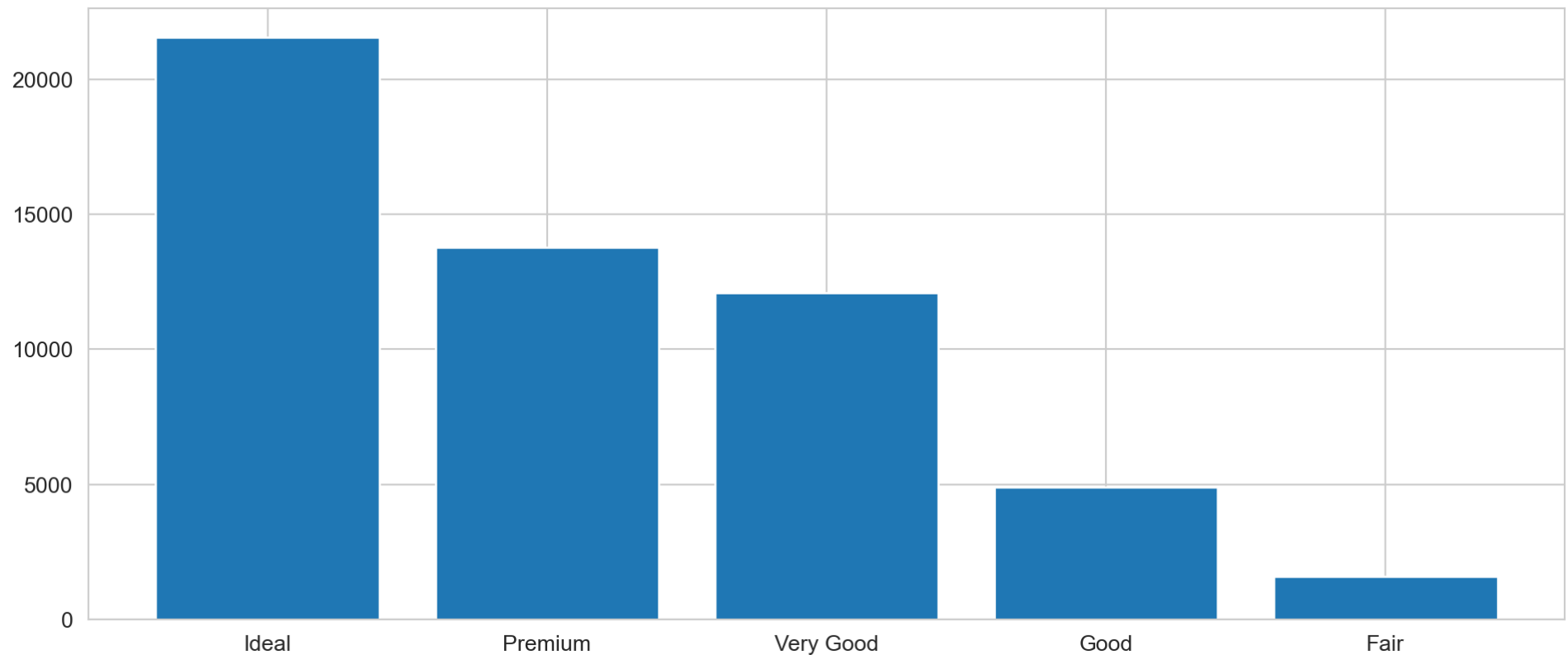
```
1 import plotly.express as px
2
3 counts_df = counts.to_frame().reset_index()
4 px.pie(counts_df, values="count", names="cut")
```



Bar chart - Matplotlib

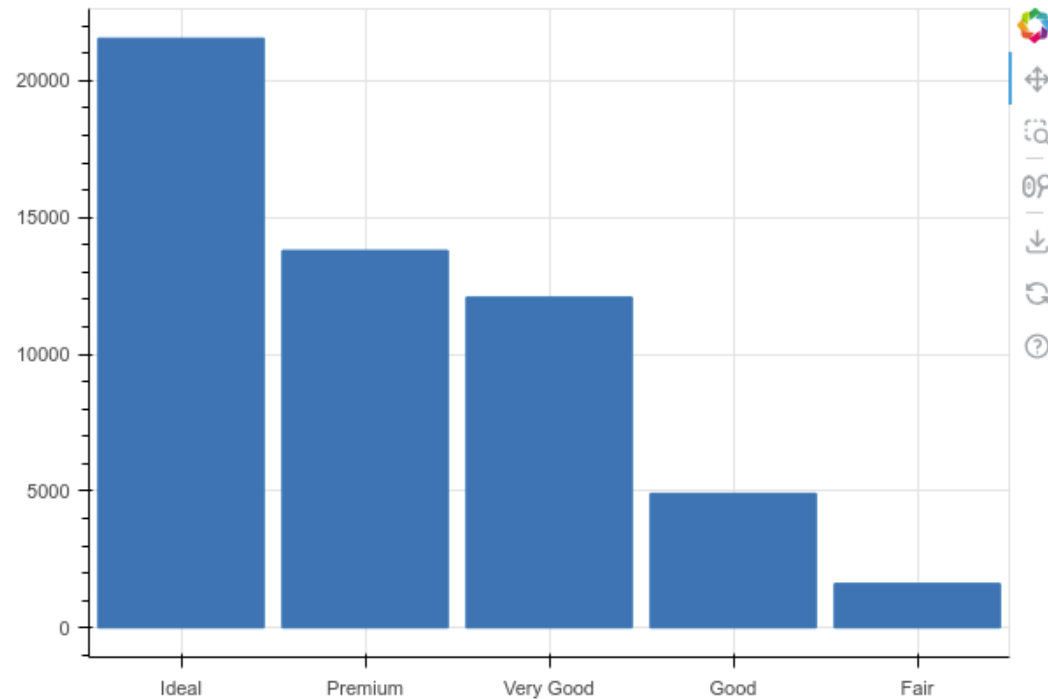
Adapté aux variables qualitatives.

```
1 fig, ax = plt.subplots(figsize=(12, 5))  
2 ax.bar(counts.index, counts);
```



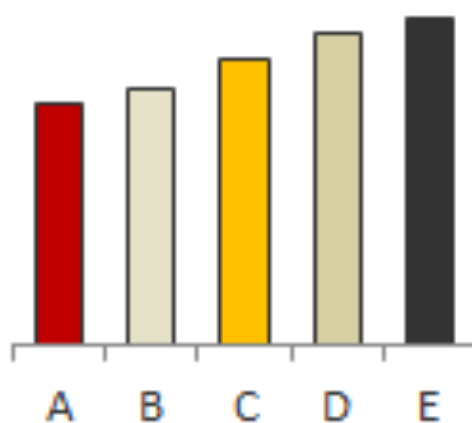
Bar chart - Bokeh

```
1 import bokeh.plotting as bp
2
3 fig = bp.figure(x_range=counts_df.cut)
4 fig.vbar(x=counts_df.cut, top=counts_df["count"], width=0.9)
5 bp.show(fig)
```

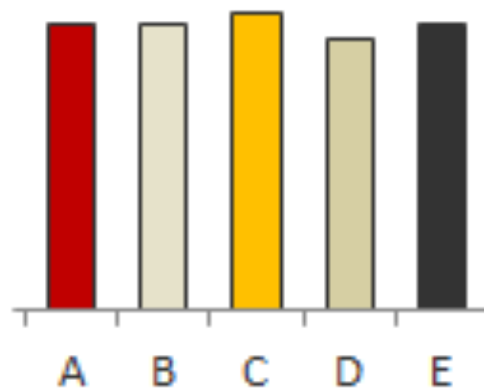


Bar Charts are Easier to Interpret than Pie Charts

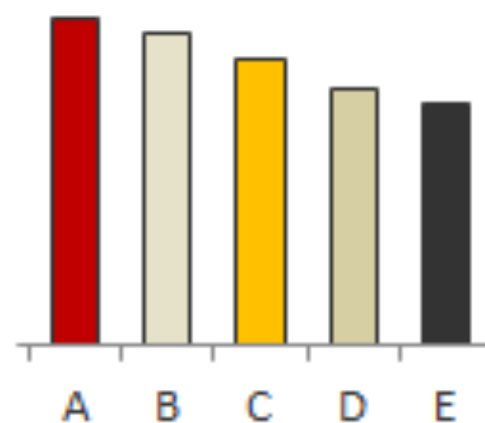
Question 1



Question 2

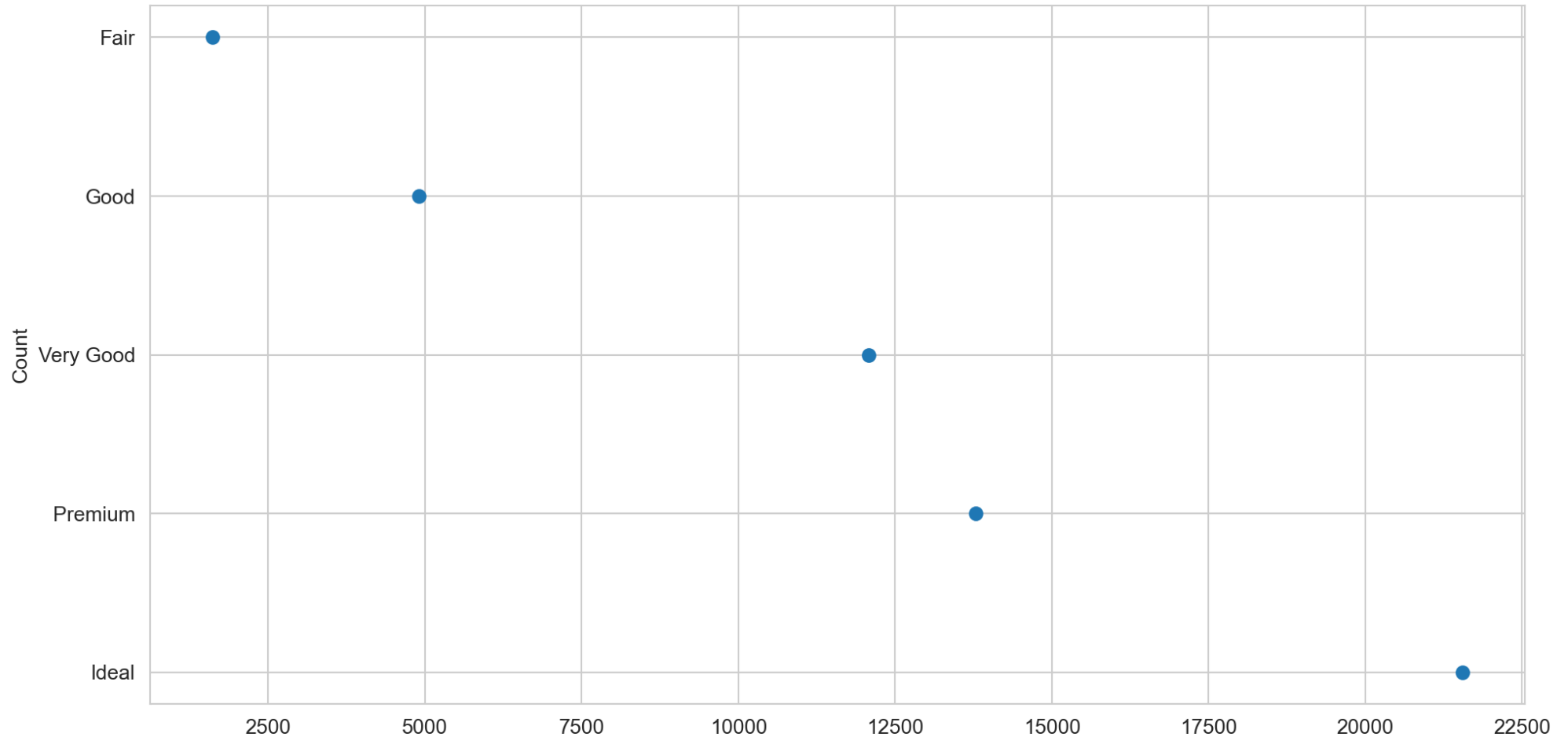


Question 3



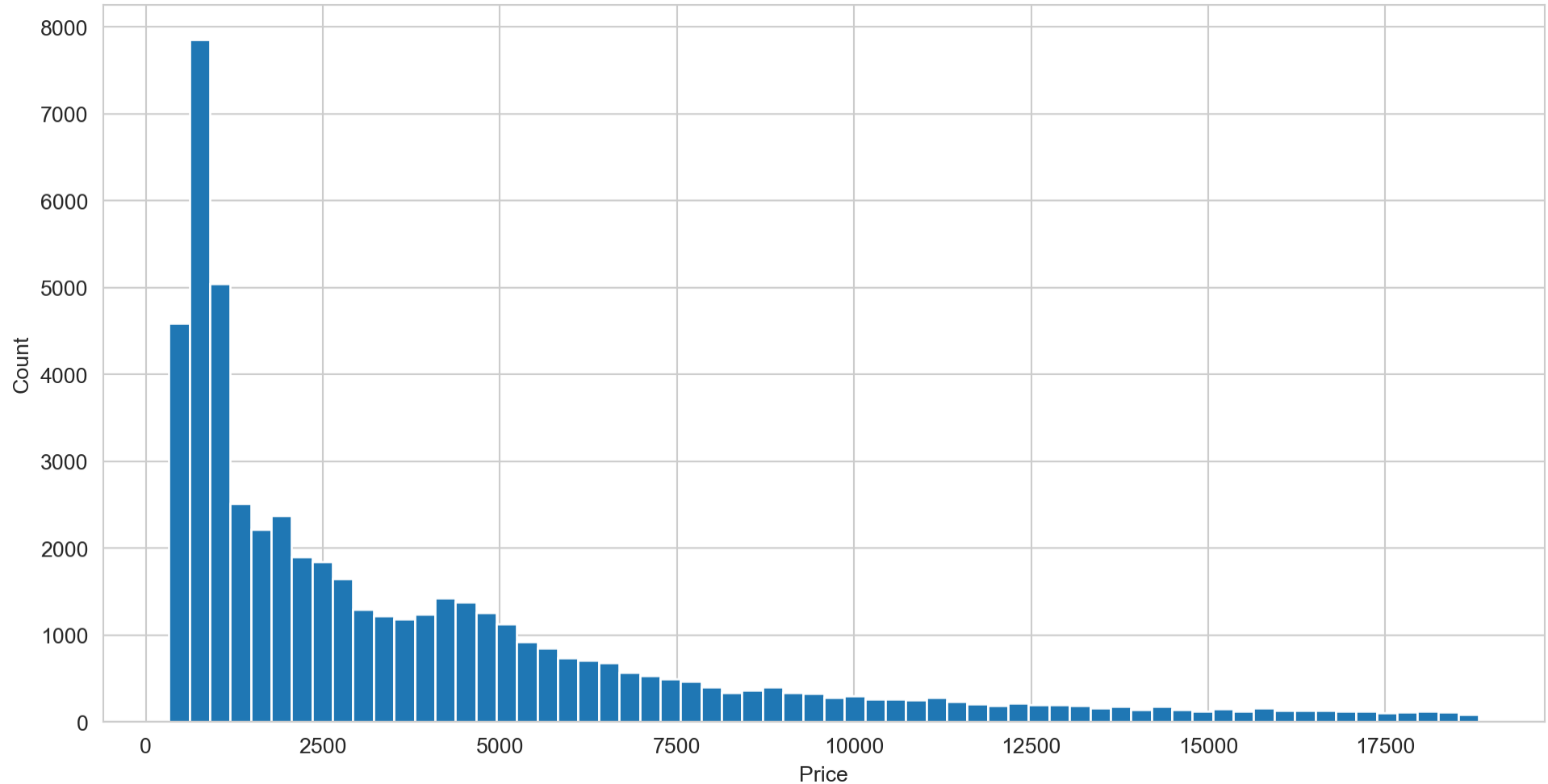
Dot plot (Cleveland)

Less ink, more pleasant...



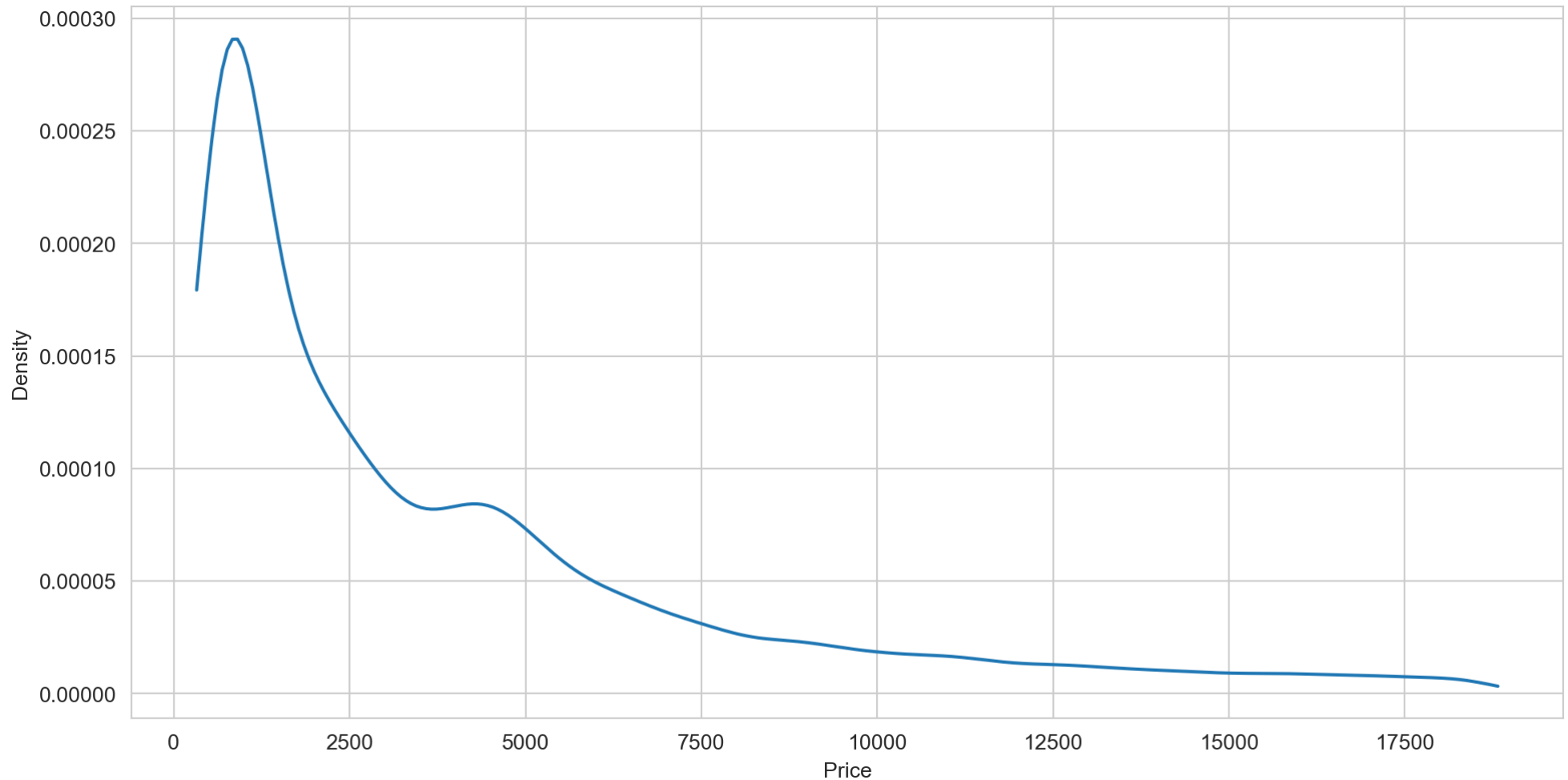
Histogramme

Adapté aux variables quantitatives.



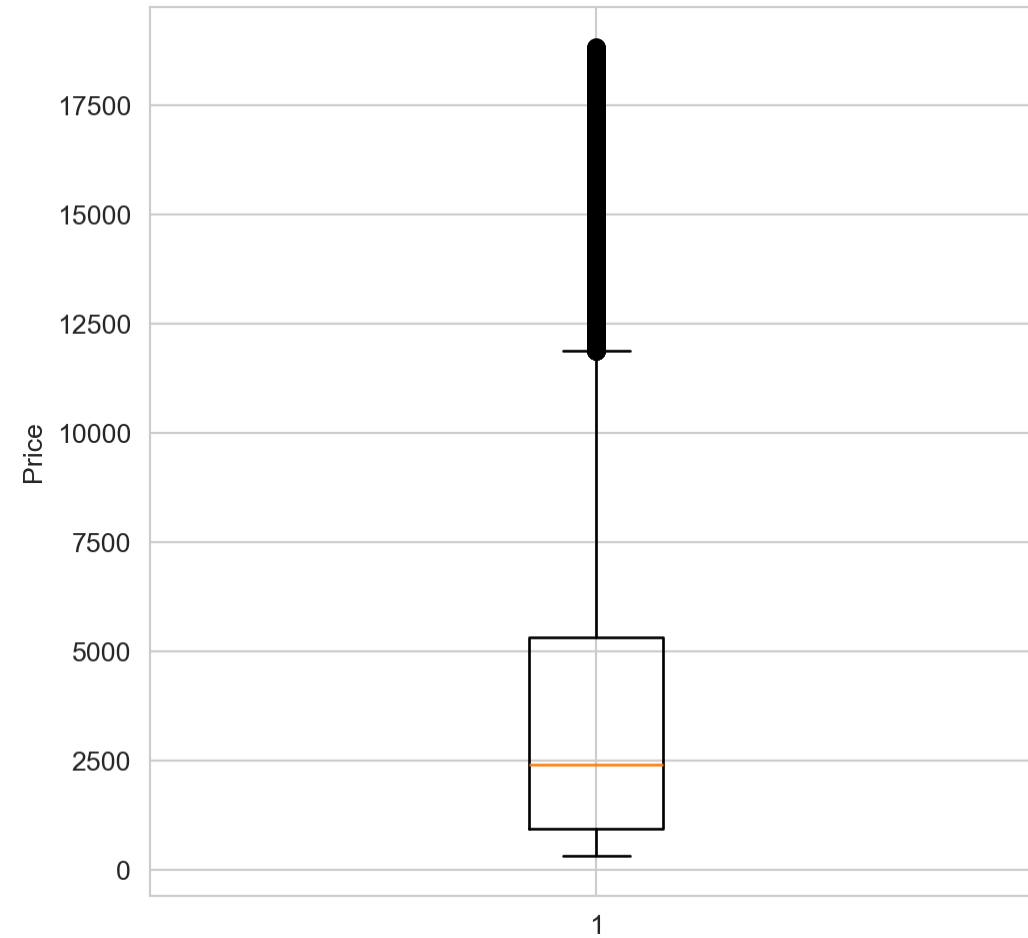
Estimateur de la densité

Version lissée !



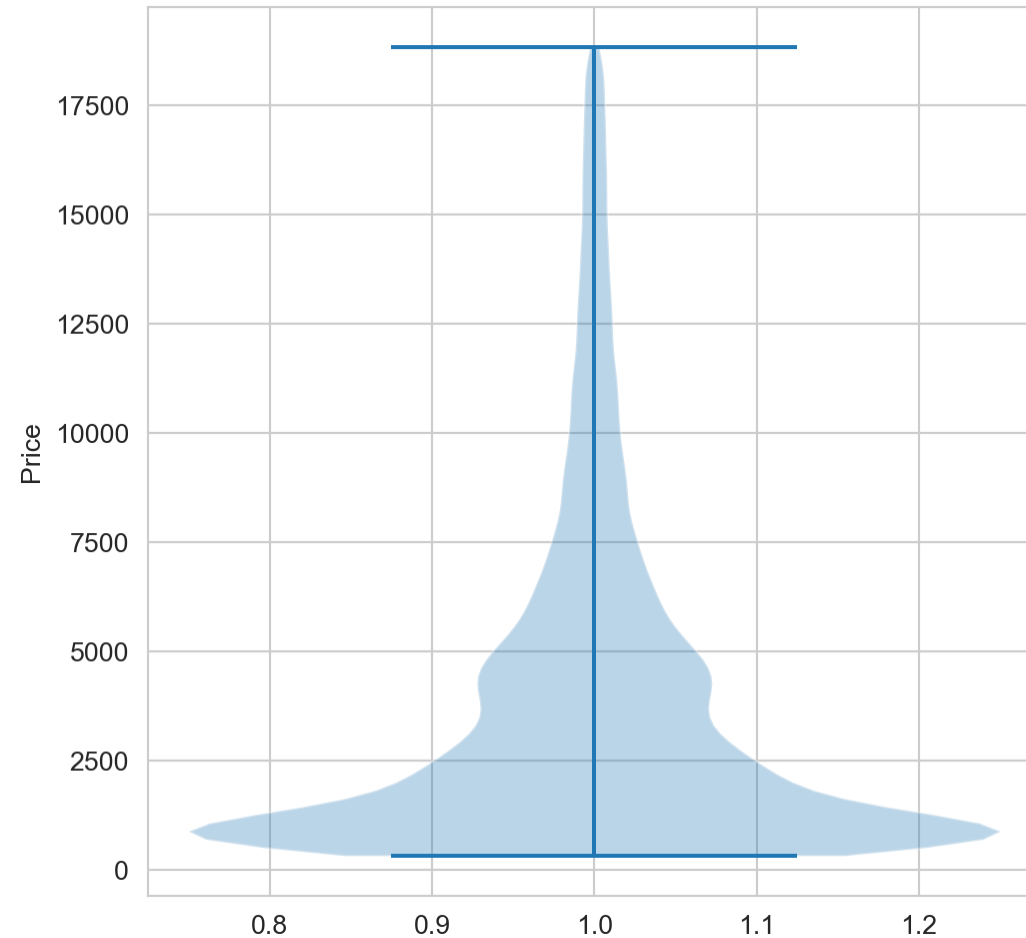
Box plot

Le grand classique !



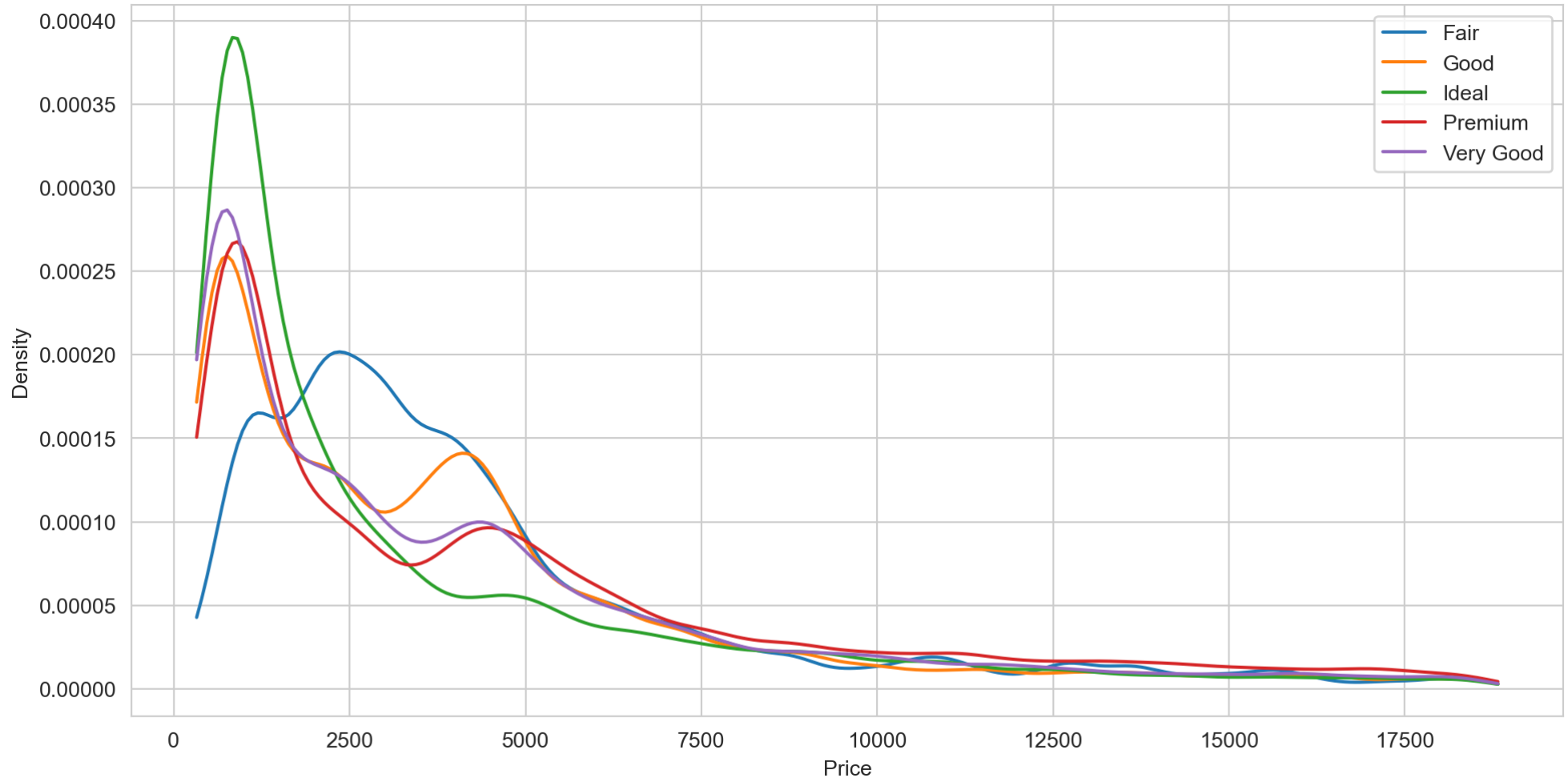
Violin plot

Combine le box plot avec l'estimation de la densité.



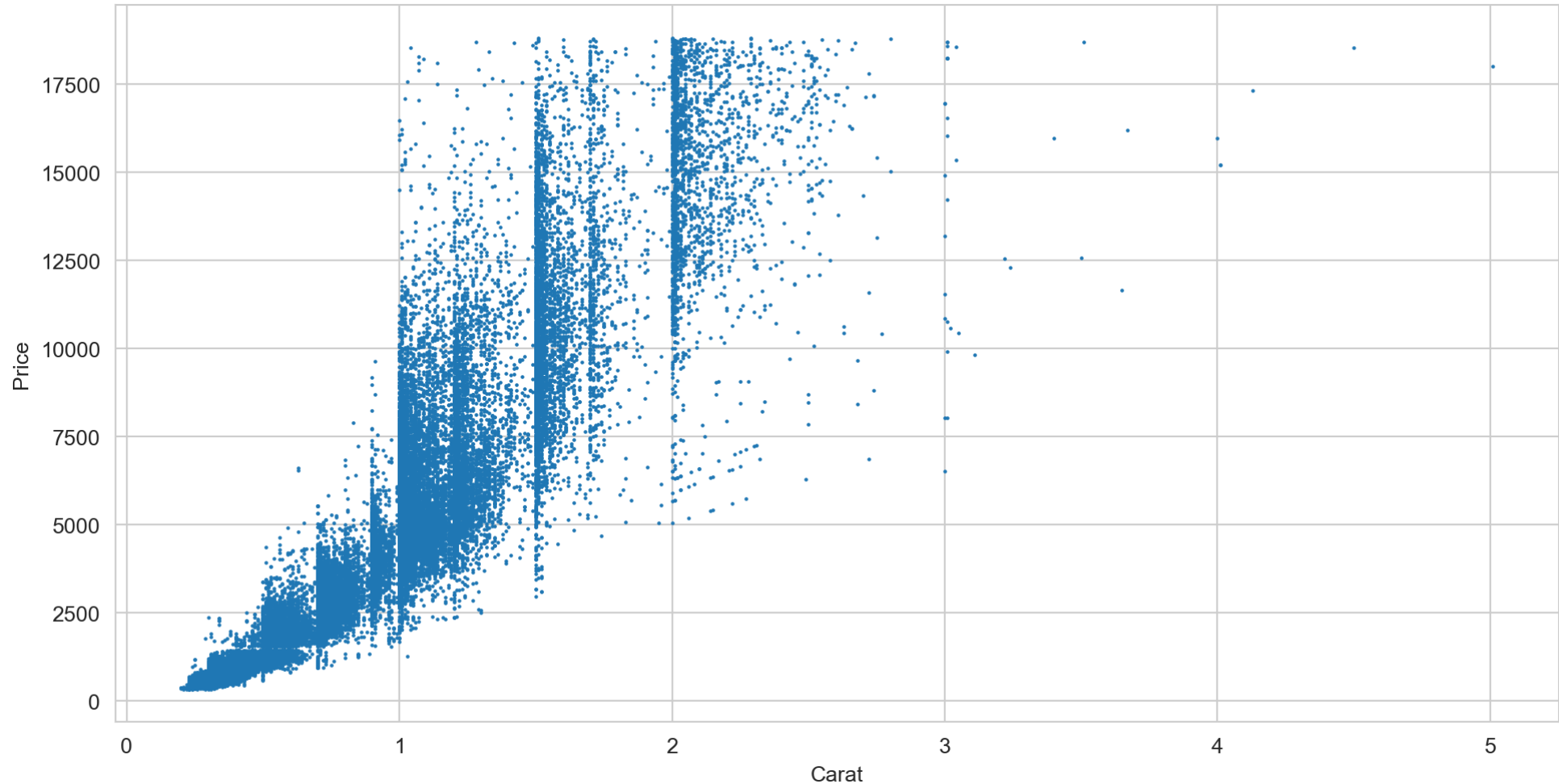
Mélange

La clé pour une représentation plus complexe.



Scatter plot (nuage de points)

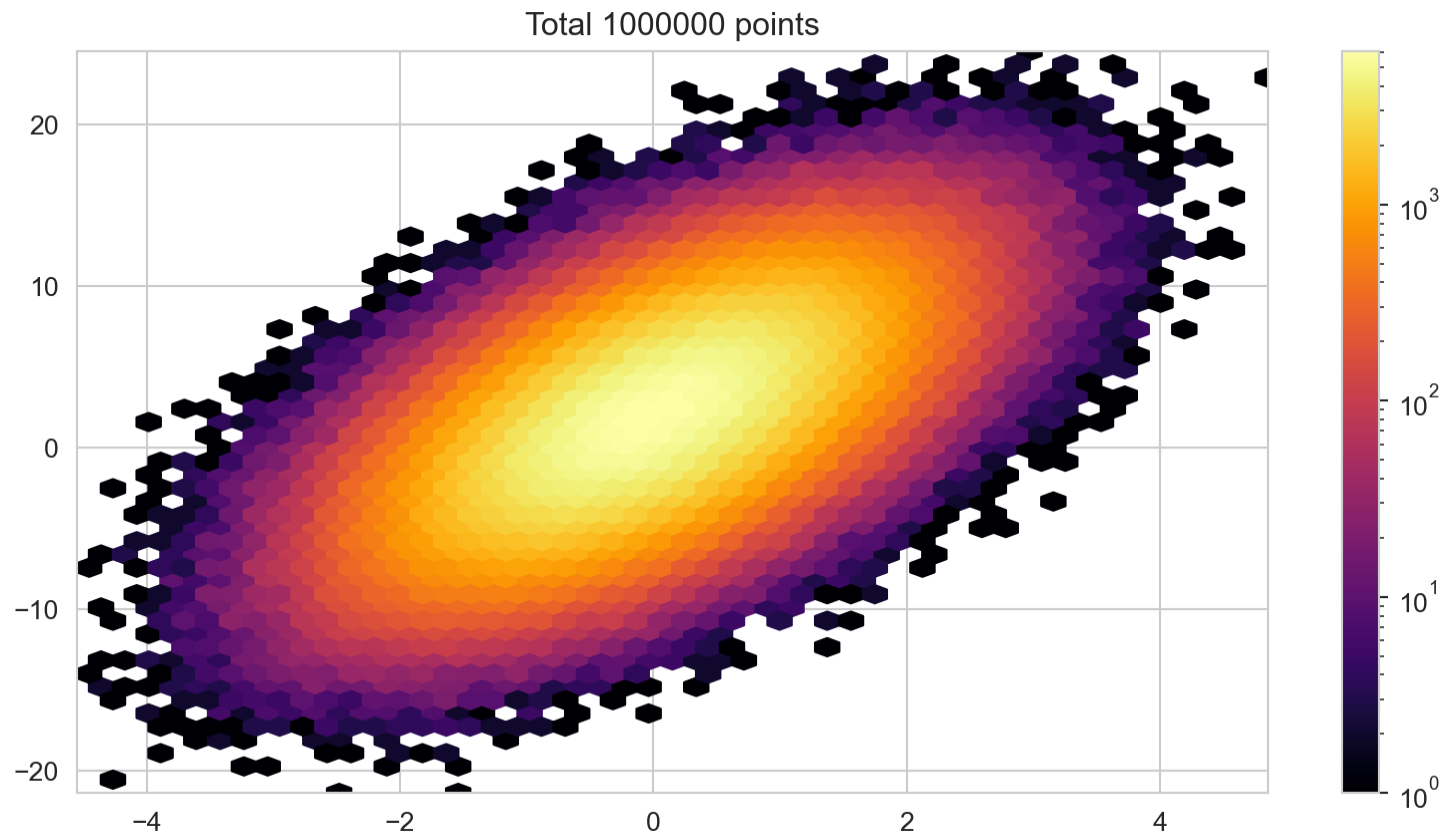
Pratique pour visualiser des relations.



Binning

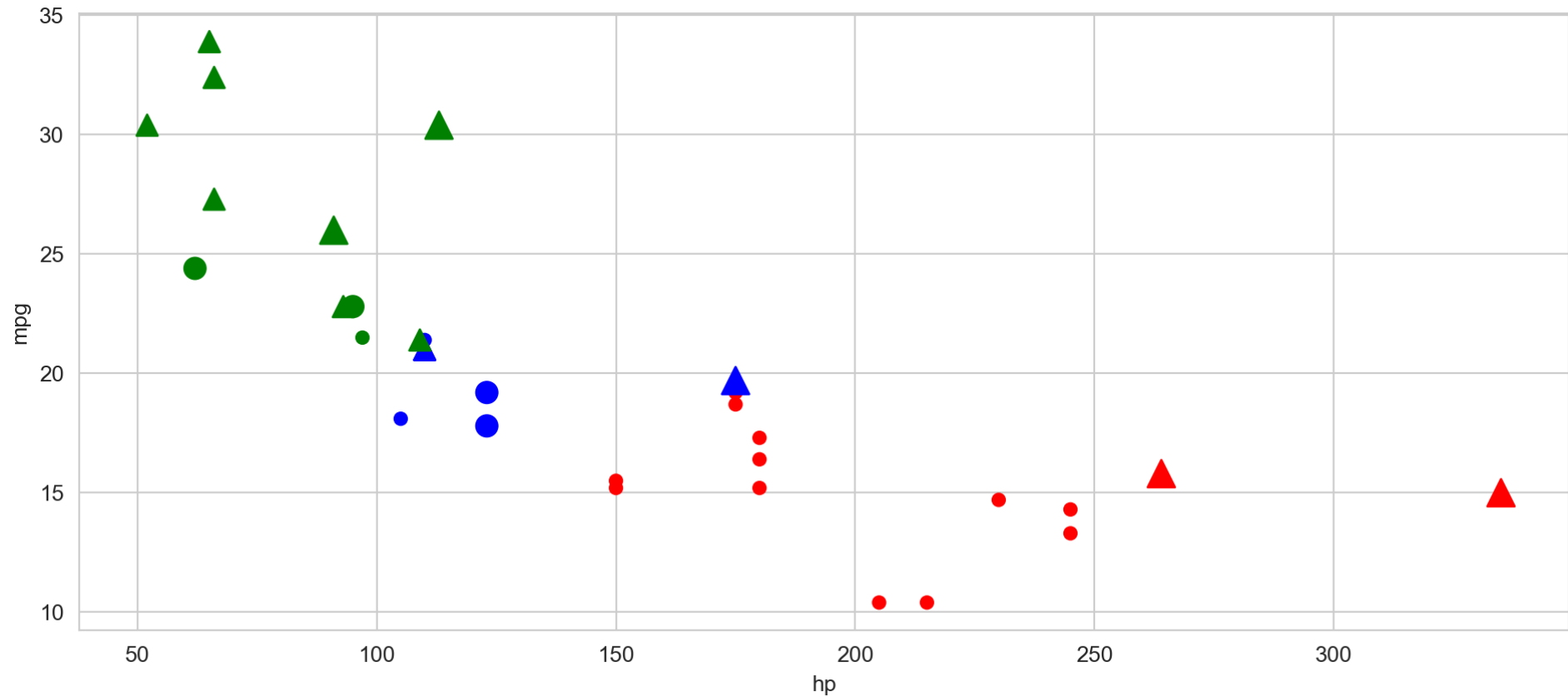
Plus de points que de pixels ! Big Data ?

Il faut résumer l'information.

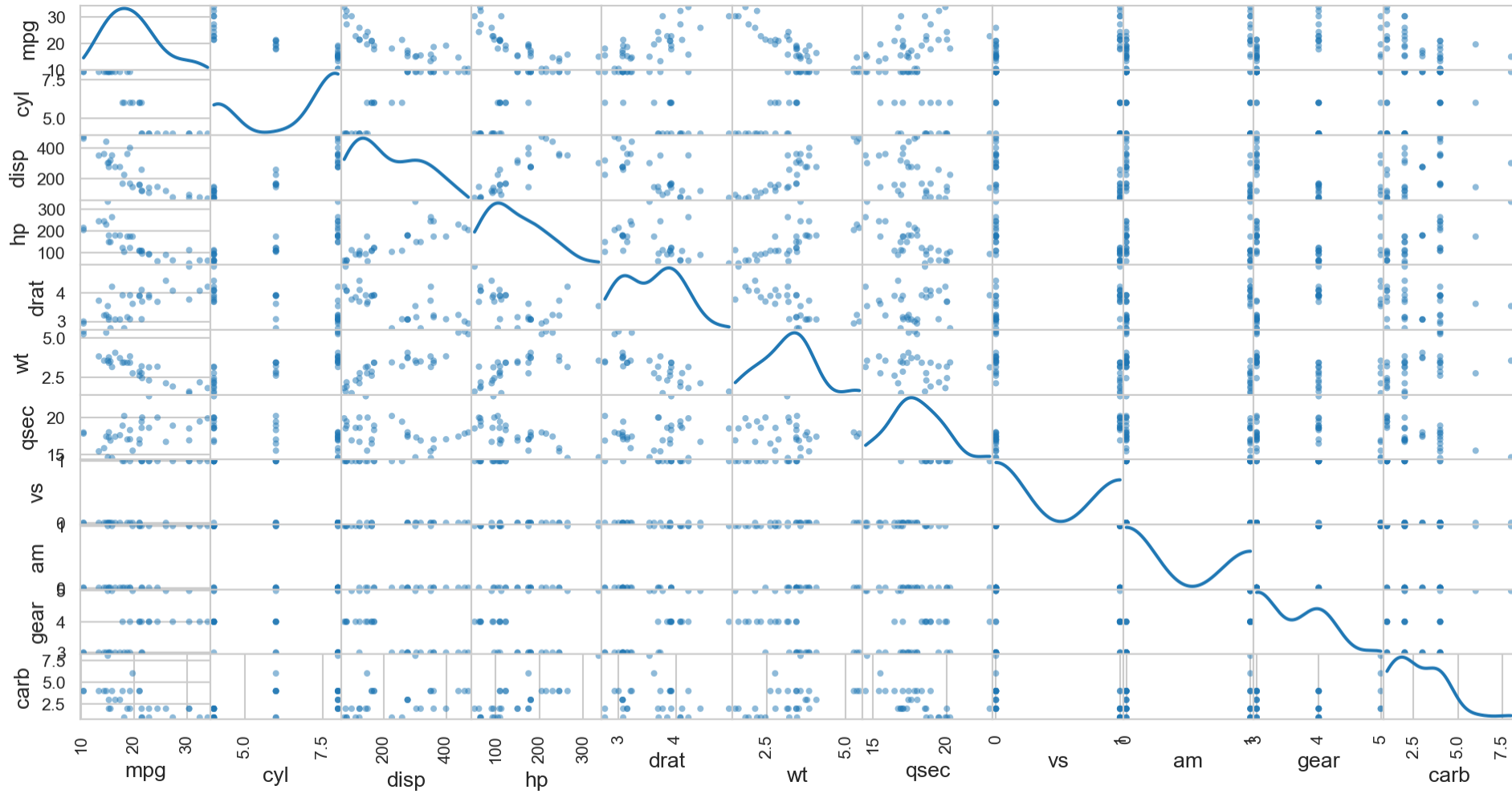


Décoration

Augmente l'information mais aussi la complexité.

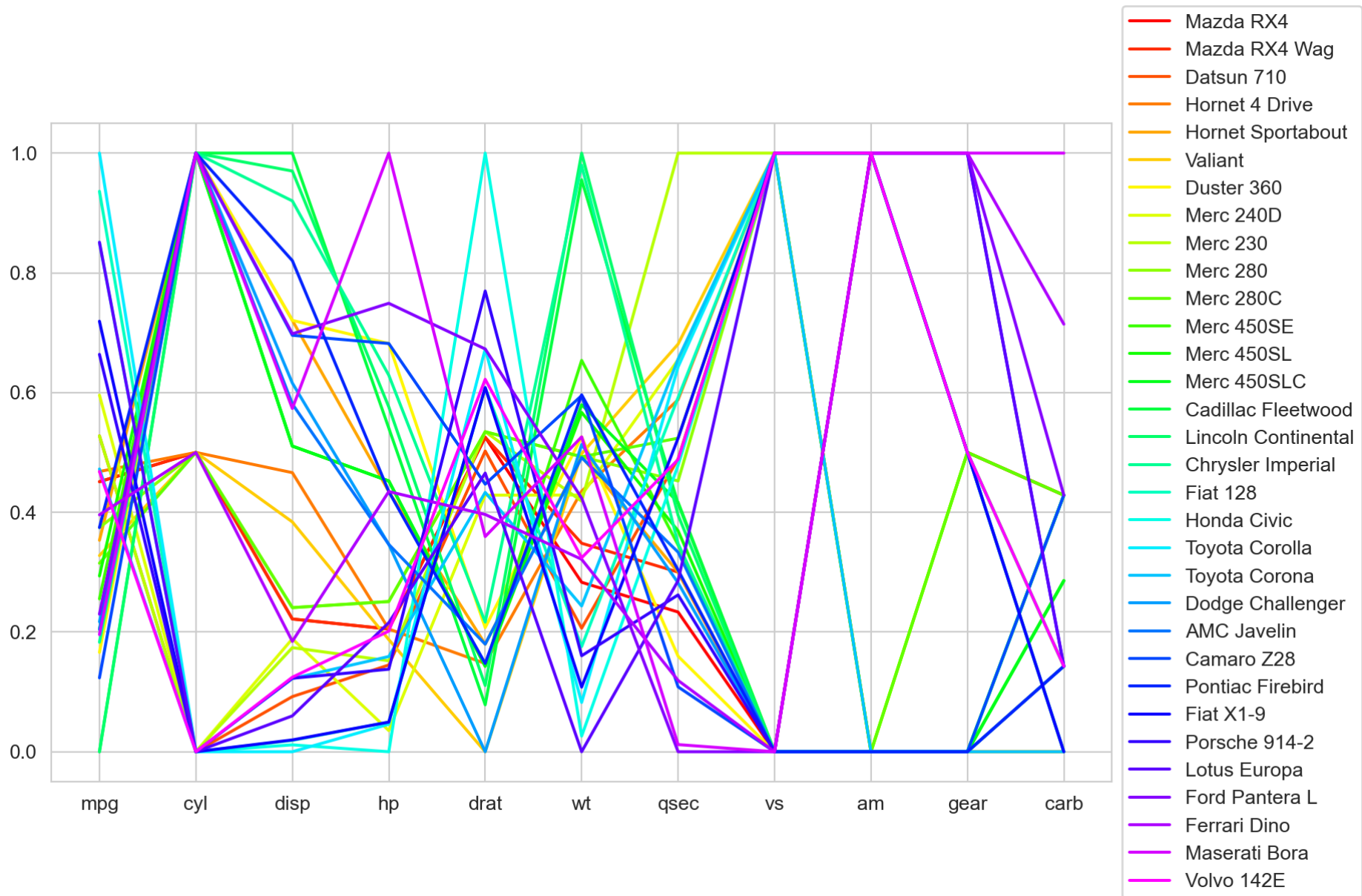


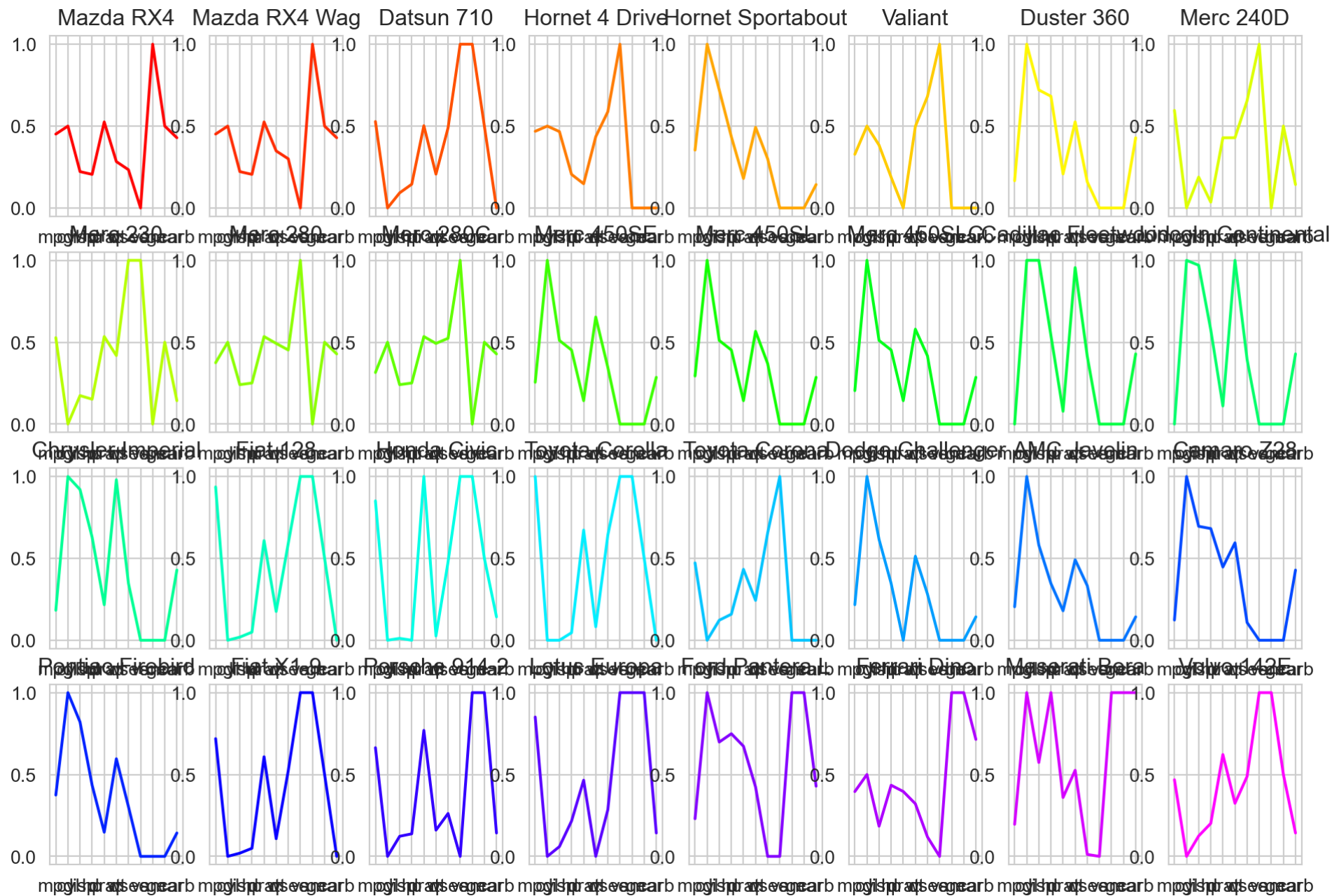
Scatter plot matrix



Parallel Coordinates

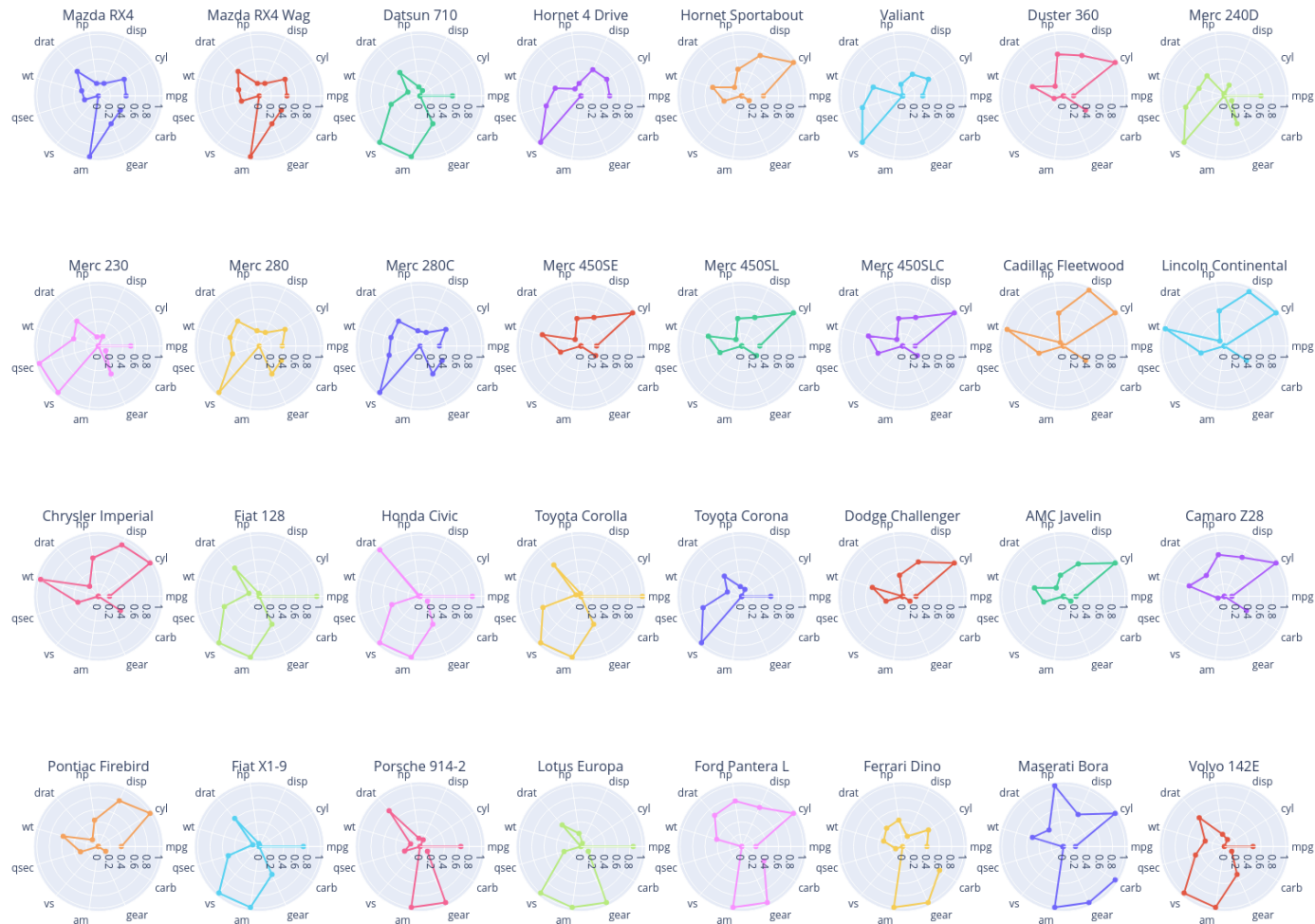
Intéressant pour visualiser des groupes.





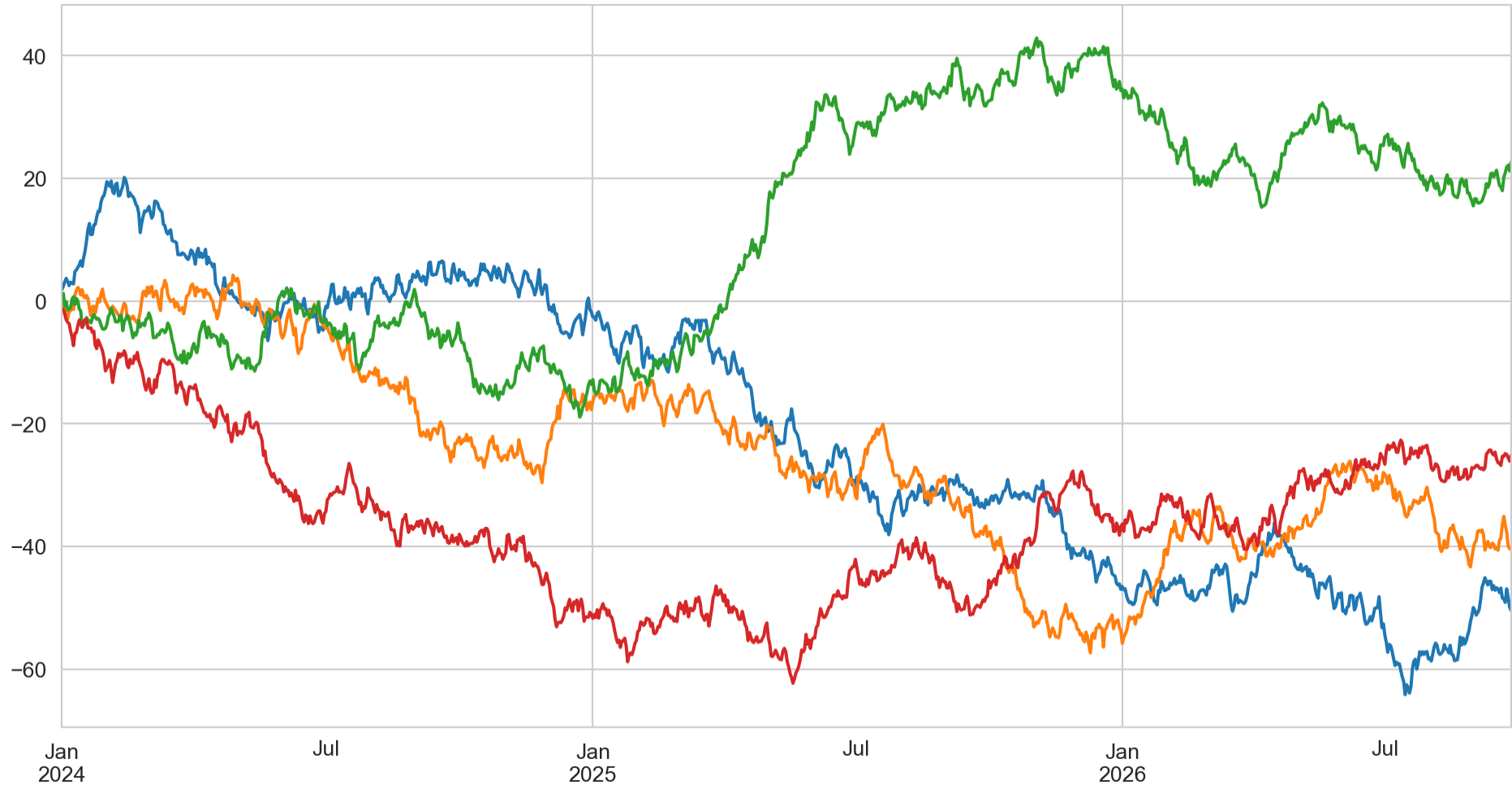
Radar plot

Merci Florence Nightingale



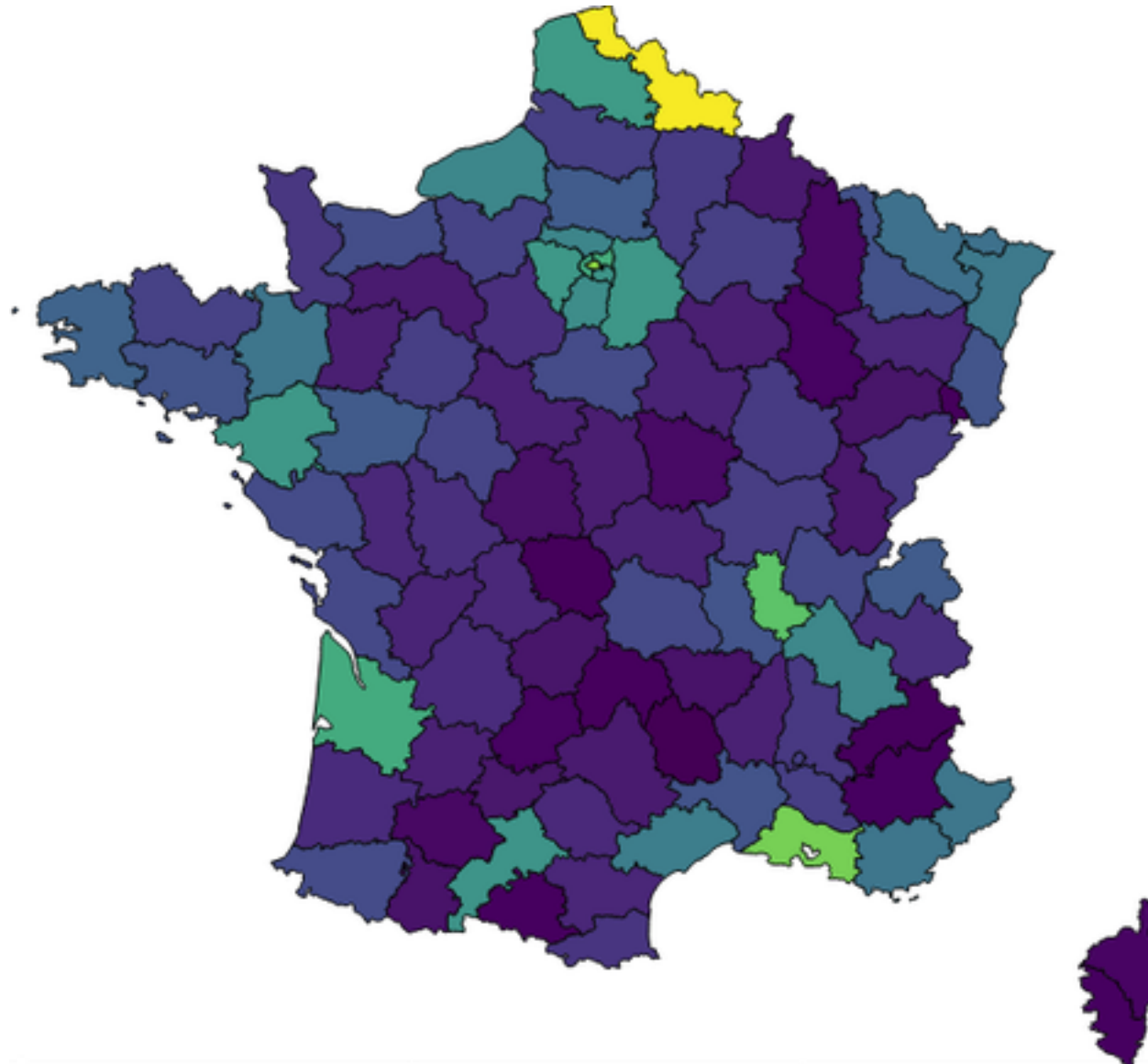
Séries temporelles

L'ordre rend la ligne pertinente.



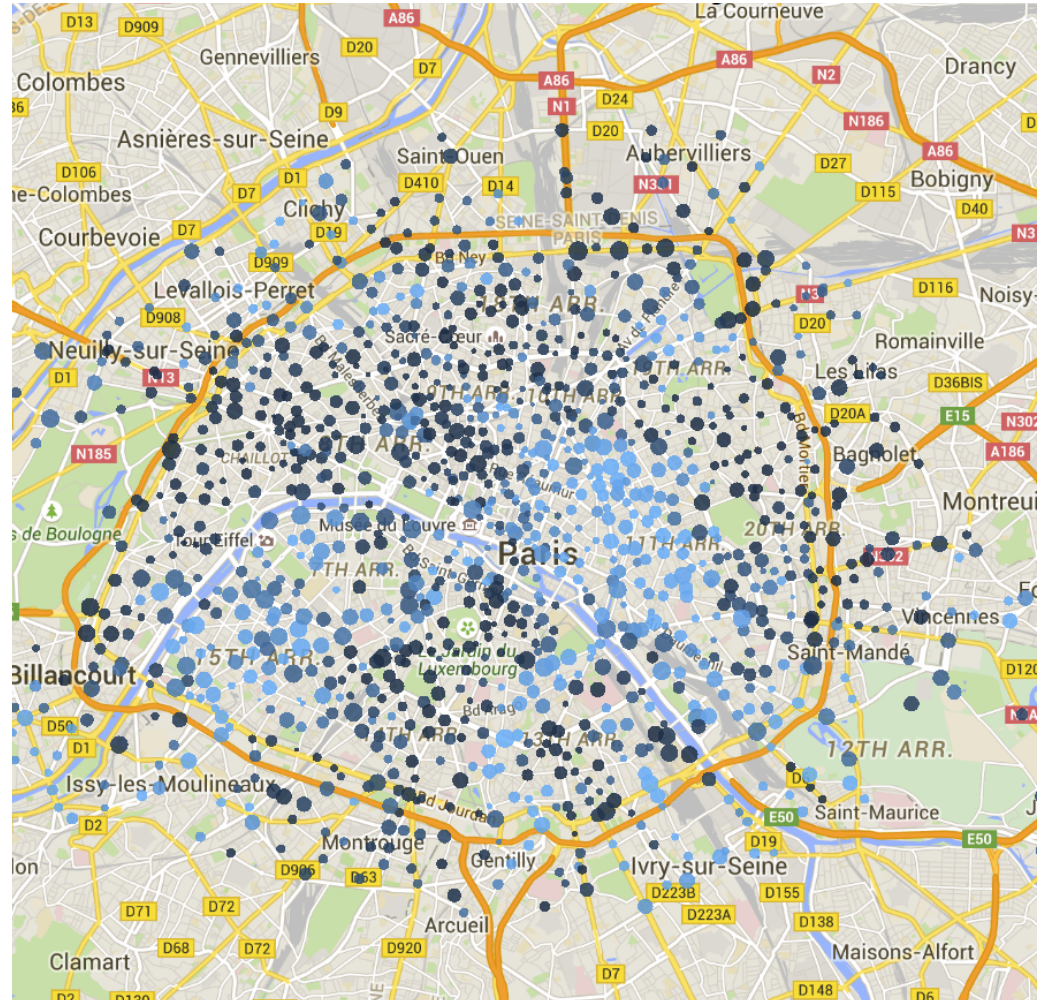
Choropleth

Impact visuel fort !



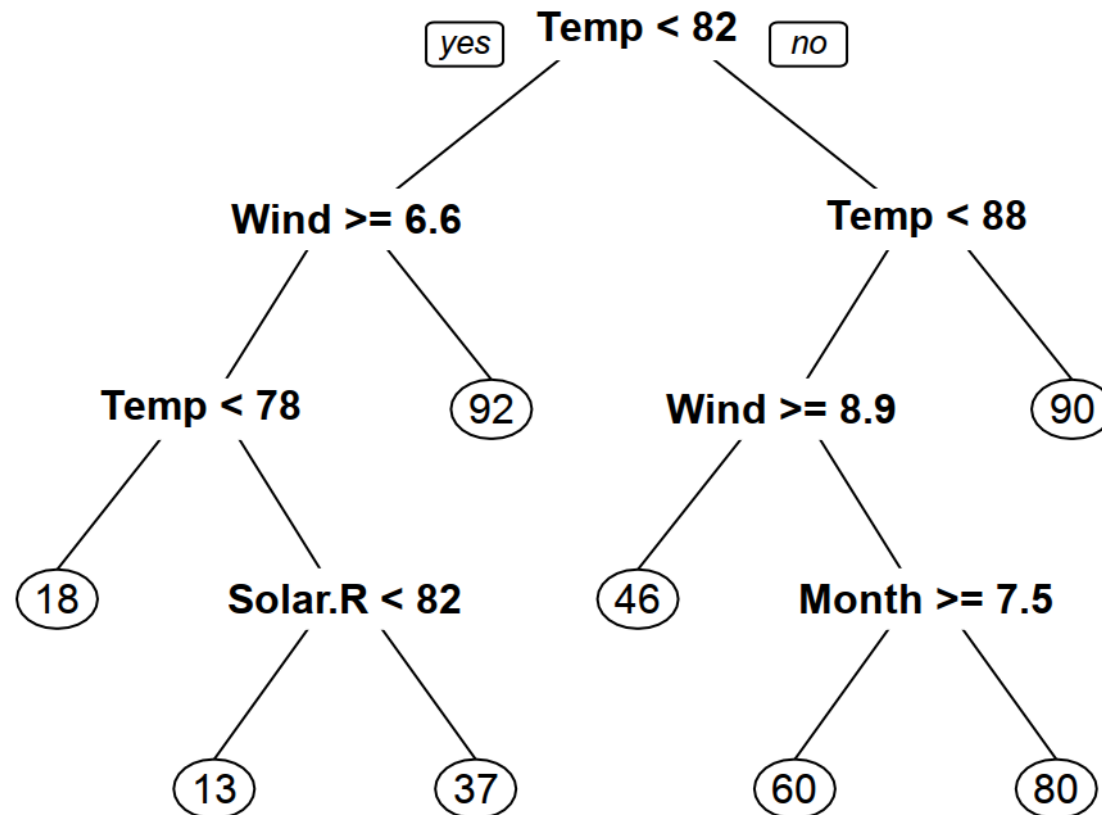
Symbol map

Peut être étendu à des décorations plus complexes.



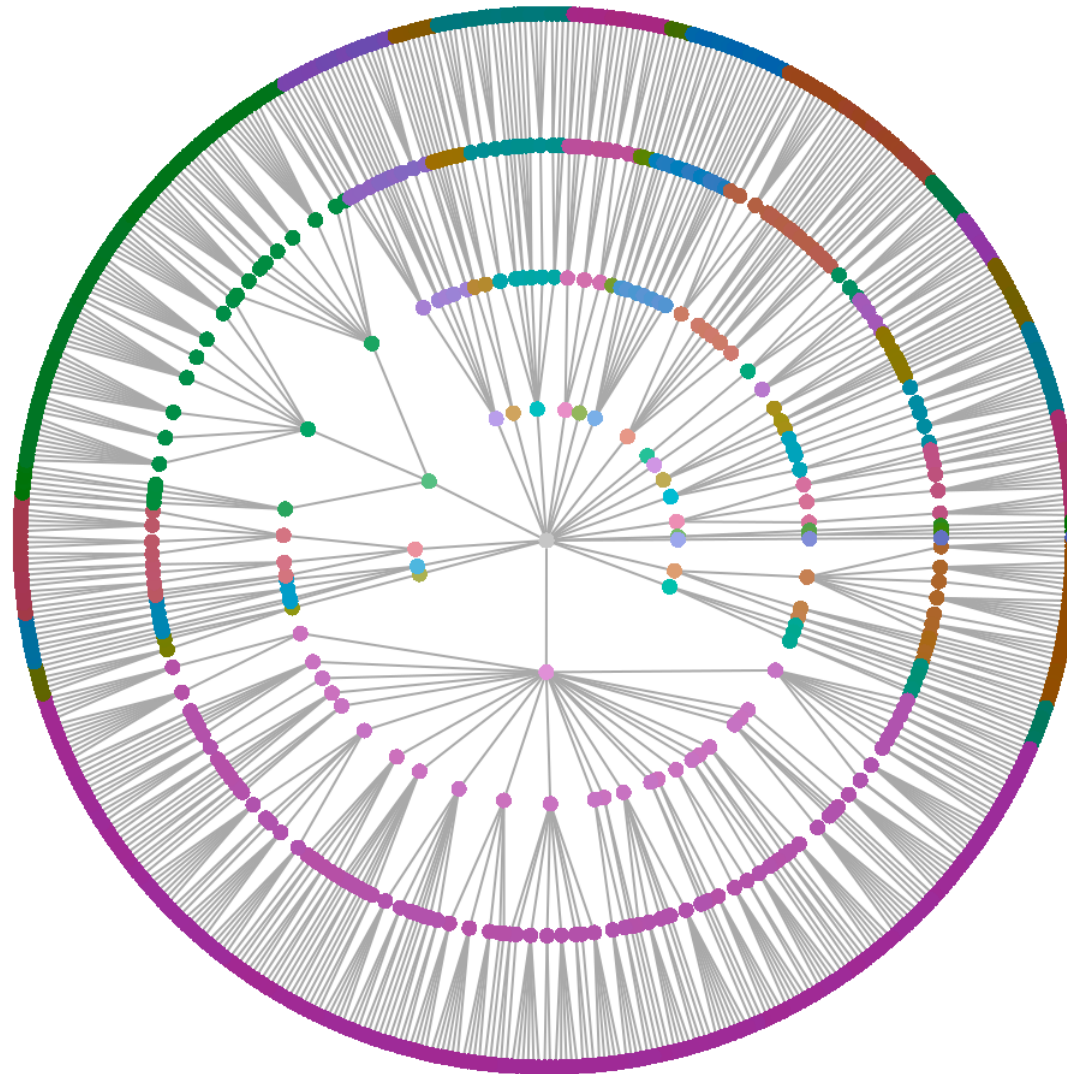
Arbre

Très utilisé en discrimination (classification supervisée).



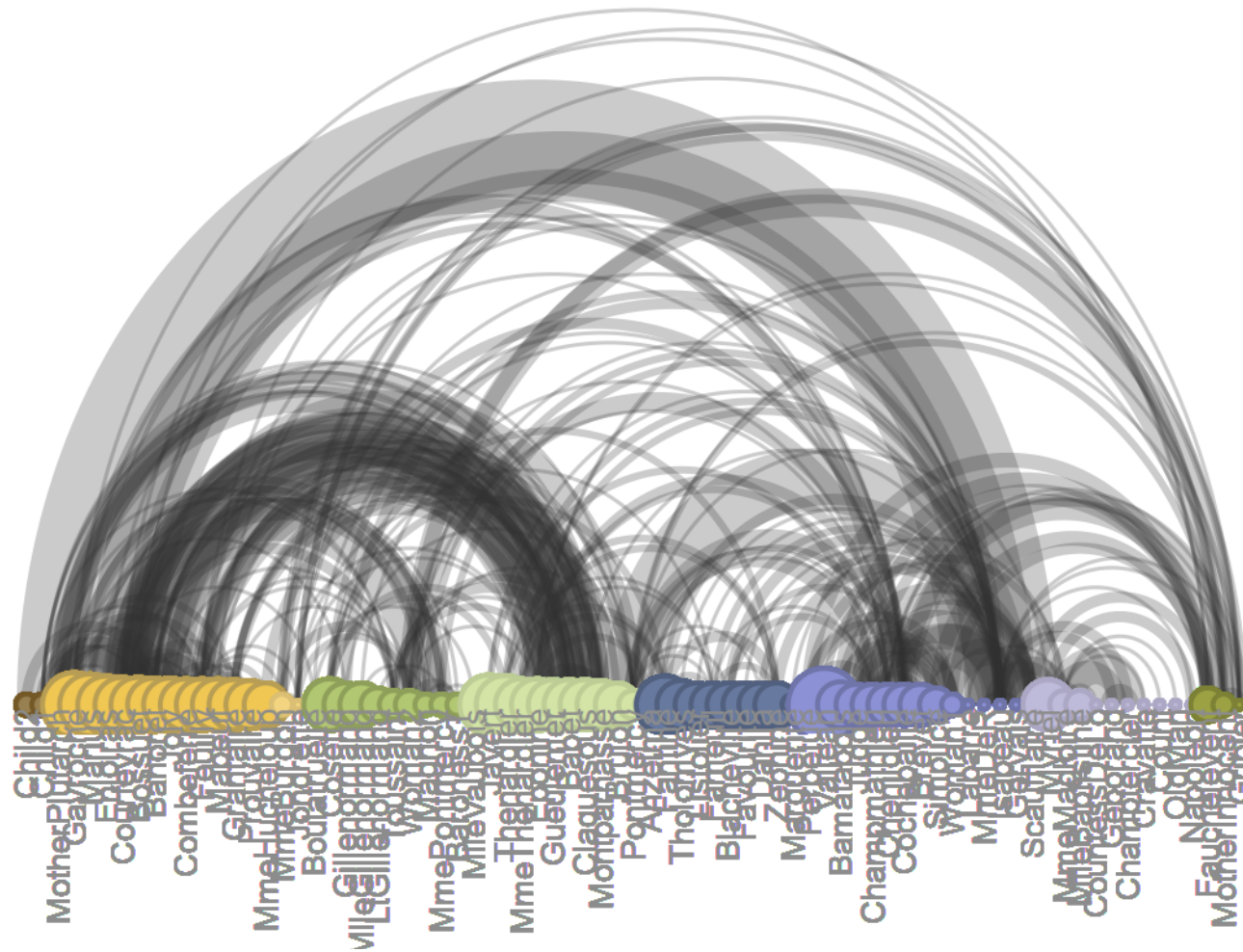
Arbre circulaire

Une variante des arbres.

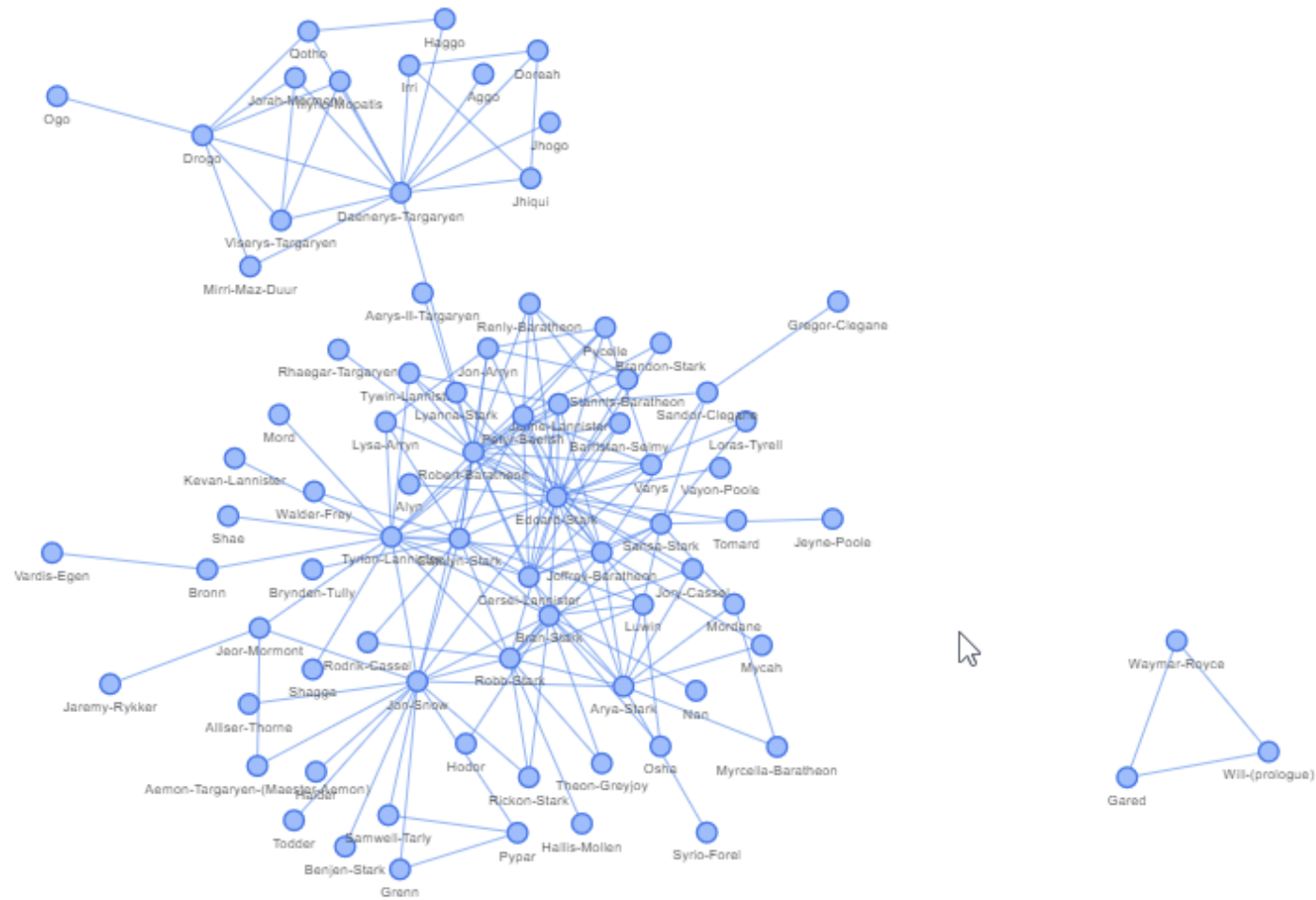


Arc diagram

Visualisation d'un graphe.

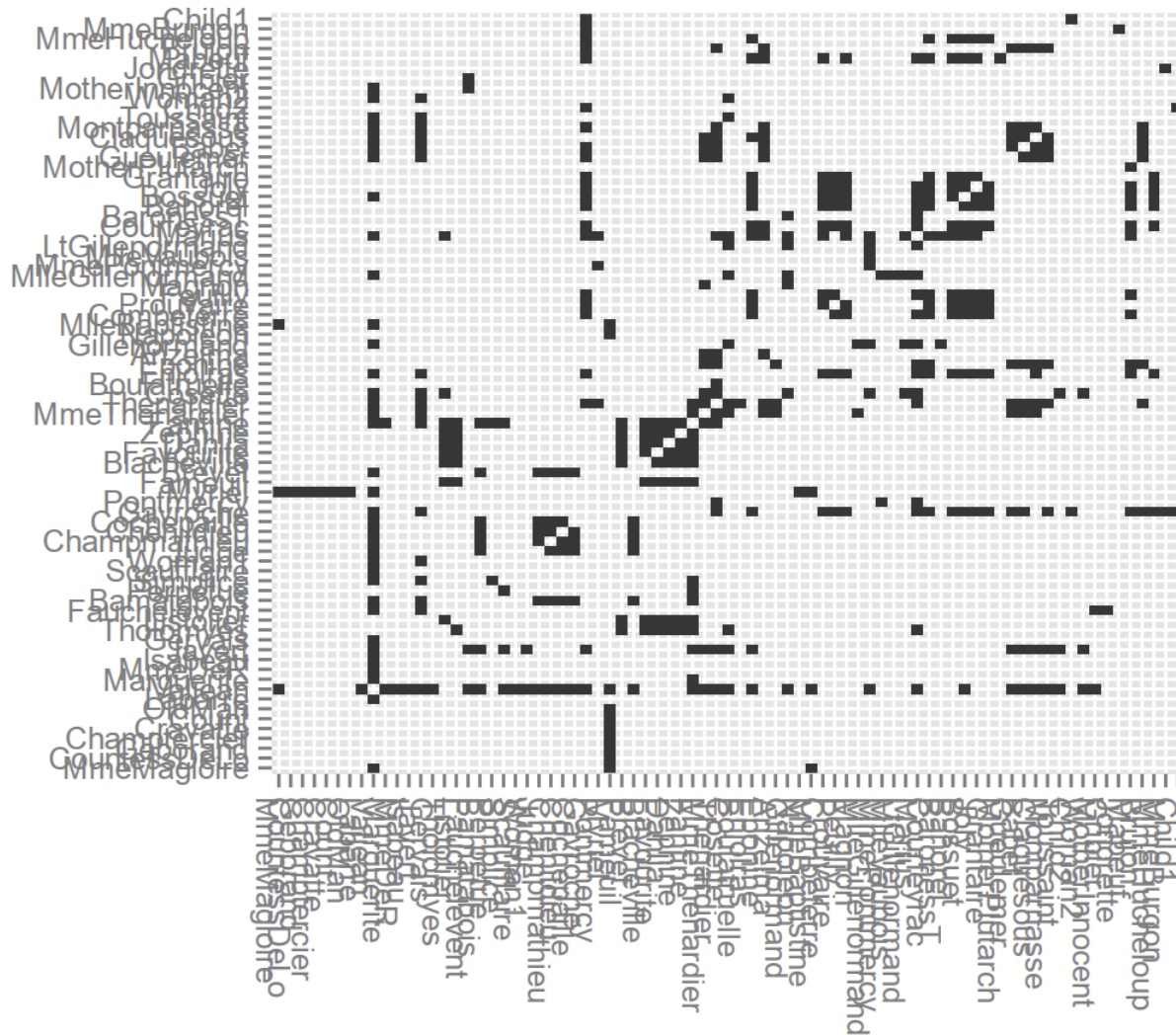


Network graph

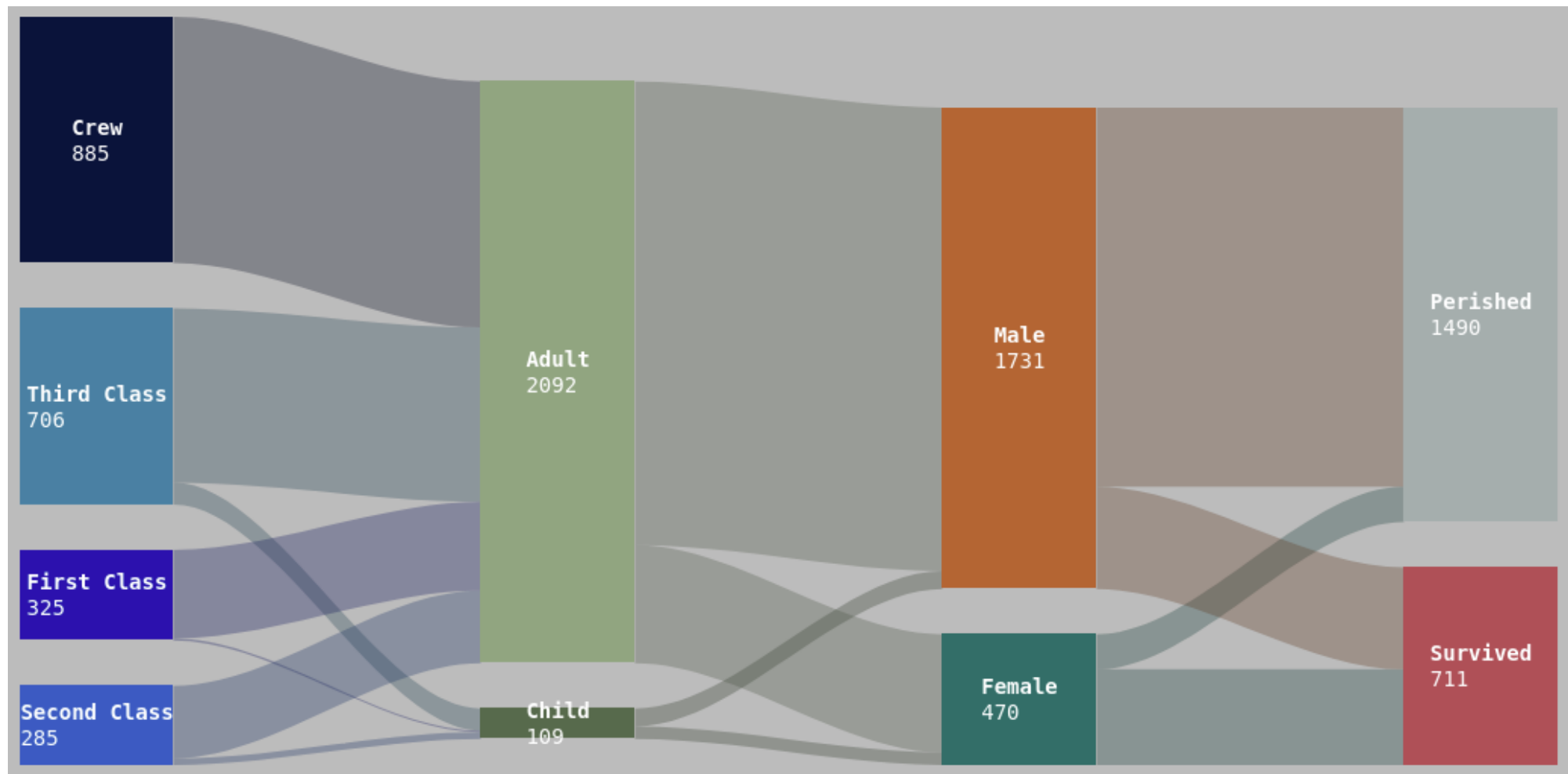


Adjacency matrix

Visualisation par la matrice d'adjacence.

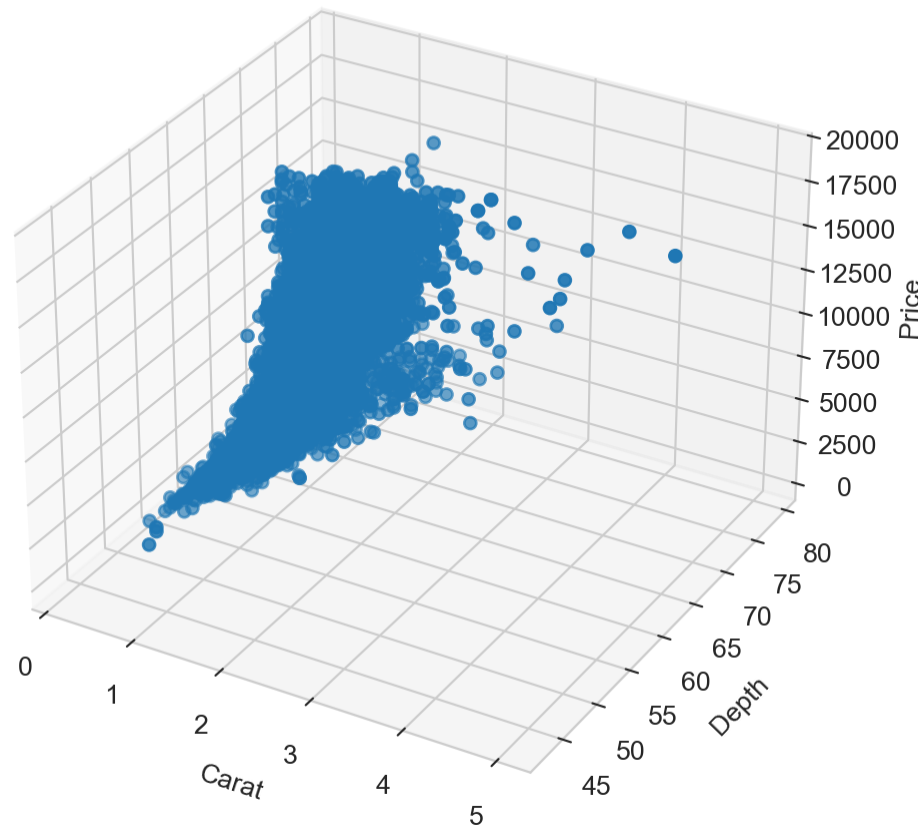


Sankey diagram



3D

Les visualisations 2D se généralisent (scatter plot, ...)



Visualisations interactives

Grâce à des bibliothèques JavaScript, les dashboards interactifs et les applications web permettent de communiquer efficacement :

- en augmentant l'information (apportée, pas celle disponible),
- en proposant l'interactivité (scroll, zoom, ...),
- en augmentant l'intérêt pour un public non spécialiste.

Cela transforme la représentation statistique usuelle en la rendant attractive, originale et facile à inclure dans divers médias.

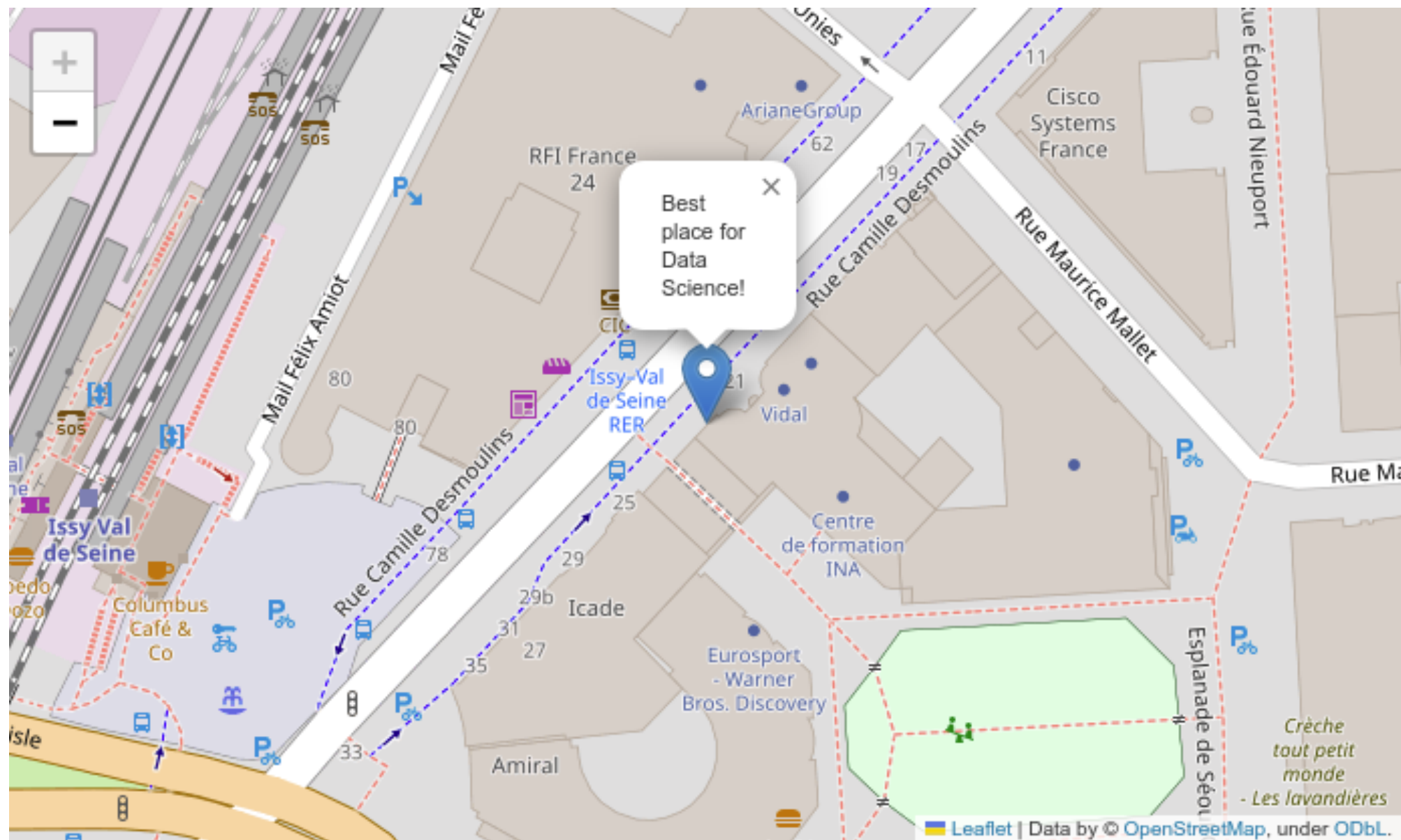
Quelques modules Python

- `bokeh`, `plotly` : visualisation interactive,
- `altair`, `pyecharts` : graphiques interactifs,
- `ipywidgets` : interactivité dans les notebooks Jupyter,
- `folium` : cartes,
- `pydeck` : cartes en 3D,
- `pyvis` : graphes, réseaux et bien plus,
- ...

Outils adaptés pour des applications web (`panel`, ...) comme nous allons le voir 😎

Exemple d'une carte

```
1 import folium
2
3 cepe = (48.8297938, 2.2649786)
4 m = folium.Map(location=cepe, zoom_start=18)
5 folium.Marker(
6     location=cepe, popup="Best place for Data Science!"
7 ).add_to(m)
8
9 m
```



Évolution permanente,
Restez curieux !